

# Technologies web dynamiques avec JavaScript avancé



G. Cherradi 2025

[https://www.researchgate.net/profile/Ghyzlane\\_Cherradi](https://www.researchgate.net/profile/Ghyzlane_Cherradi)



# PLAN

- Introduction (Web application, JavaScript avancé)
- Manipulation avancée du DOM et des événements
- Modules ES6 et Structuration du Code
- Programmation asynchrone (Callbacks, Promises, Async/Await)
- Programmation Orientée Objet (POO) en JavaScript
- Manipulation de Données avec JSON/XML et APIs
- API Web, AJAX et Fetch
- Introduction à React.js (Front-end moderne)
- Introduction à Nest.js (Back-end JavaScript)
- Optimisation et Sécurité



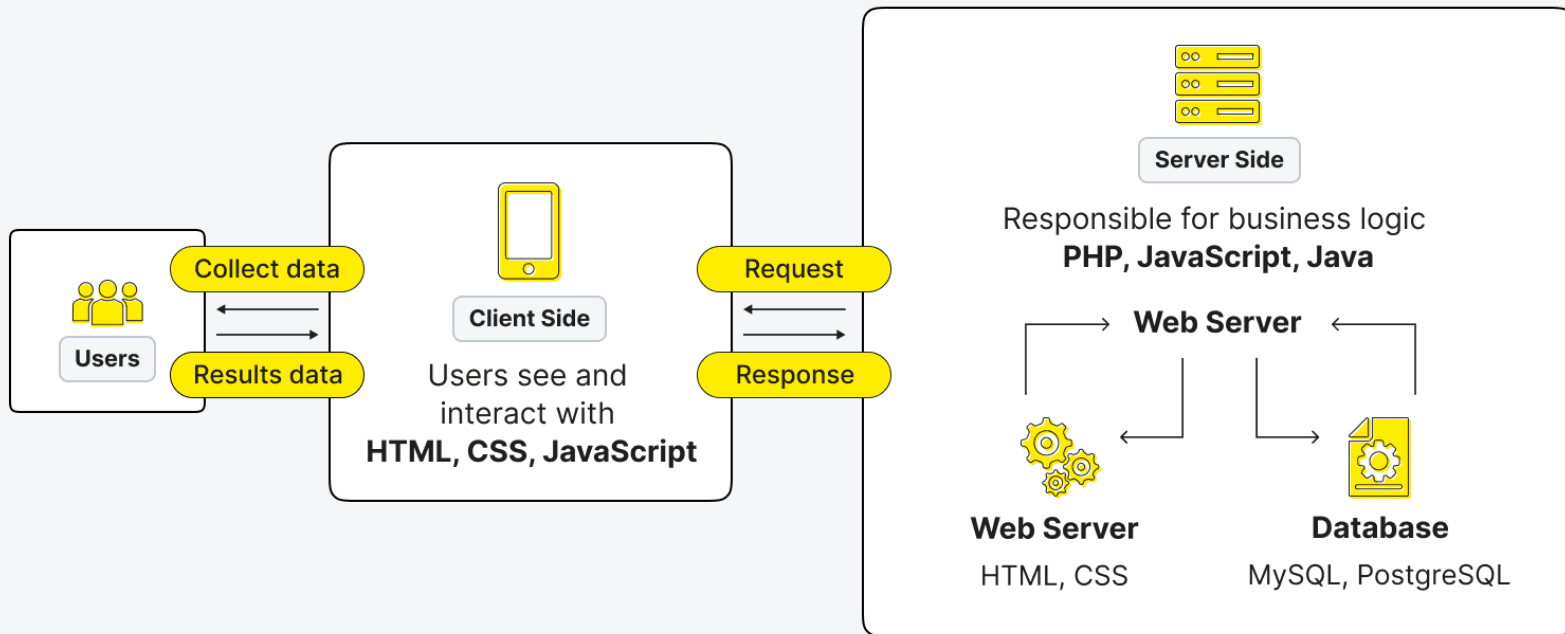
Knowledge Check (Quiz) + Badges



<https://www.youracclaim.com/organizations/ibm/badges>

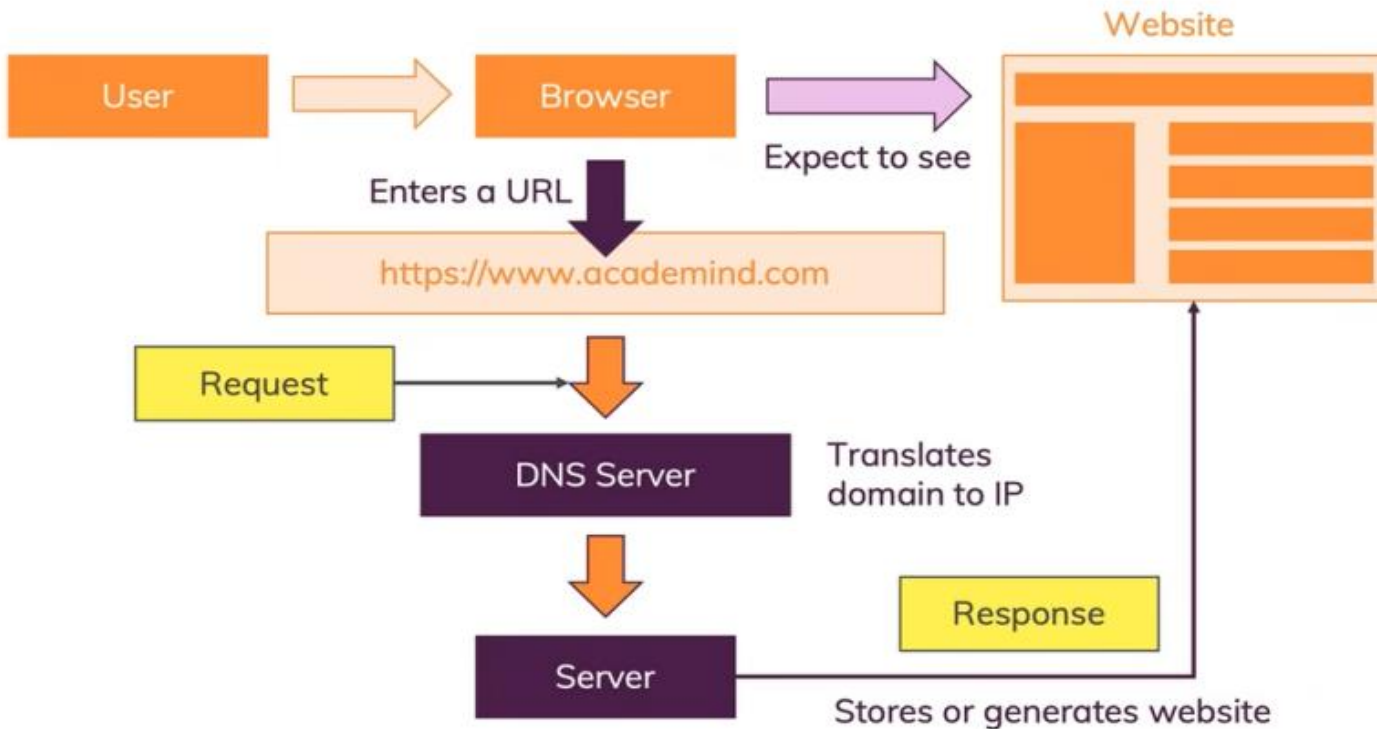
# Introduction au langage JavaScript

## System Architecture for a Web Application



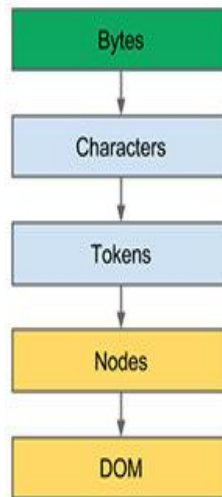
# Introduction au langage JavaScript

## How The Web Works



# Introduction au langage JavaScript

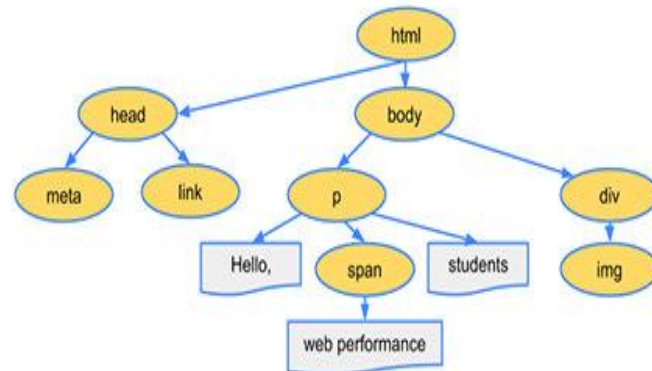
1. Le navigateur télécharge le fichier HTML sous forme de bytes
2. À l'aide de l'encodage (UTF-8), le navigateur transforme les octets en caractères lisibles.
3. Le navigateur lit le texte caractère par caractère, et il le découpe en tokens (Un token, c'est un petit morceau significatif).
4. Le navigateur transforme chaque token en node (Un node, c'est un élément vivant en mémoire). Chaque node est un objet JavaScript interne.
5. Le navigateur organise tous les nodes en arbre hiérarchique (**DOM** : document object Model).



3C 62 6F 64 79 3E 48 65 6C 6C 6F 2C 20 3C 73 70 61 6E 3E 77 6F 72 6C 64 21 3C 2F 73 70 61 6E 3E 3C 2F 62 6F 64 79 3E

<html><head>...</head><body><p>Hello <span>web performance</span>...</body></html>

StartTag: html StartTag: head ... EndTag: head StartTag: body StartTag: p Hello ...

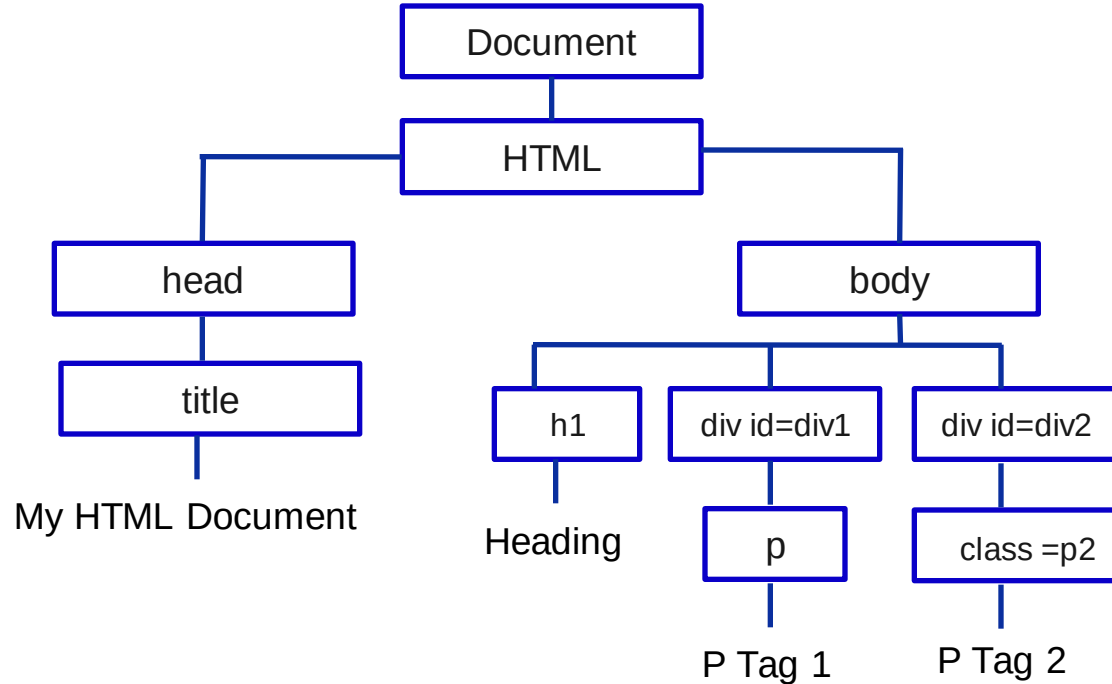


# Document Object Model (DOM)

## HTML Document

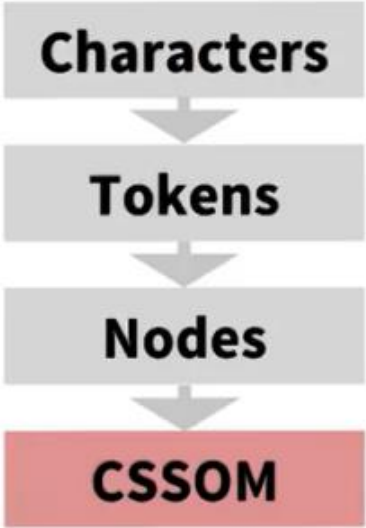
```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>My HTML Document</title>
5 </head>
6 <body>
7   <h1>Heading</h1>
8   <div id="div1">
9     <p>P Tag 1</p>
10  </div>
11  <div id="div2">
12    <p class="p2">P Tag 2</p>
13  </div>
14 </body>
15 </html>
```

## Document Object Model (DOM)

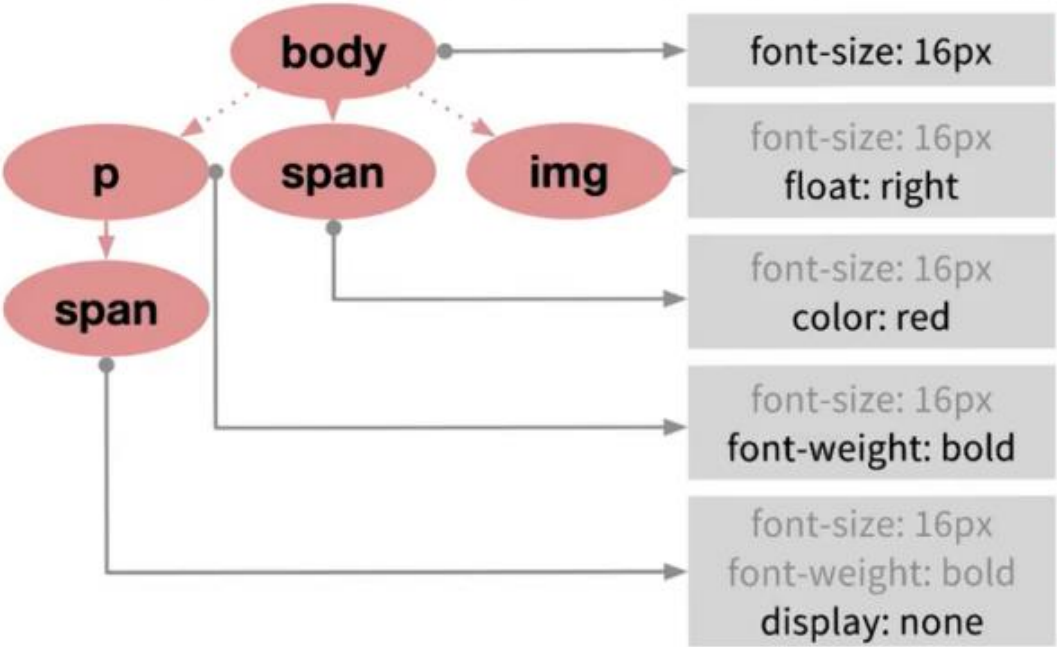


# CSS Object Model (CSSOM)

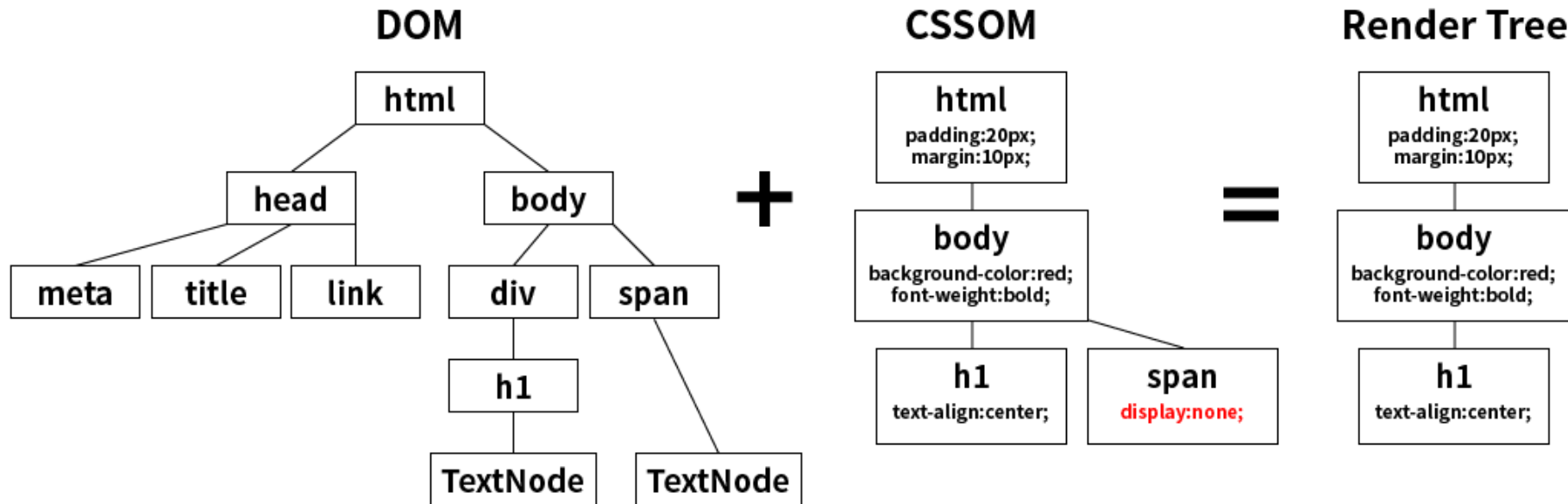
CSSOM :



```
body {font-size: 16px} p {font-weight: bold} span {color: red} p span {display: none} img {float: right}
```



# Document Object Model (DOM)



# Qu'est-ce que JavaScript ?

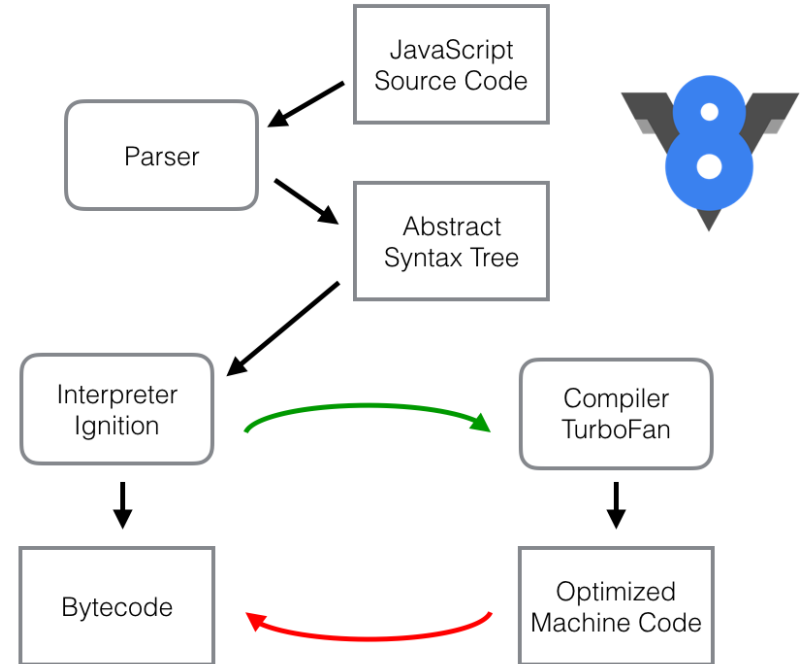
JavaScript est un **langage de programmation** utilisé pour créer du contenu dynamique pour les sites web. C'est un langage de programmation **léger, multiplateforme, mono-thread et dynamiquement typé**. JavaScript est un langage interprété qui exécute le code ligne par ligne, offrant ainsi une plus grande flexibilité.

- JavaScript est un langage mono-thread, ce qui signifie qu'il exécute une seule tâche à la fois.
- Il prend en charge à la fois le développement côté client et côté serveur
- Le type de données d'une variable est déterminé au moment de l'exécution en JavaScript.
- Polyvalent : JavaScript peut être utilisé pour une large gamme de tâches, allant de calculs simples à des applications côté serveur complexes.
- Asynchrone : JavaScript peut gérer des tâches comme la récupération de données depuis des serveurs sans bloquer l'interface utilisateur.
- Écosystème riche : Il existe de nombreuses bibliothèques et frameworks basés sur JavaScript, tels que React, Angular et Vue.js, qui rendent le développement plus rapide et plus efficace.

# Qu'est-ce que JavaScript ?

JavaScript est à la fois compilé et interprété. Le **moteur V8** améliore les performances en interprétant d'abord le code, puis en compilant les fonctions fréquemment utilisées pour accélérer leur exécution. Cela rend JavaScript efficace pour les applications web modernes.

1. **Analyseur syntaxique (Parser)** : V8 commence par analyser le code source JavaScript et le transforme en un arbre syntaxique abstrait (AST).
2. **Ignition (Interpréteur)** : V8 interprète d'abord le JavaScript à l'aide d'Ignition, un interpréteur léger. Il convertit l'AST en bytecode, une représentation intermédiaire du code. Cela permet une exécution rapide sans attendre la compilation.
3. **TurboFan (Compilateur Just-In-Time - JIT)** : Pendant l'exécution, V8 identifie les fonctions fréquemment utilisées (code chaud). Ces fonctions sont compilées en code machine hautement optimisé grâce à TurboFan. Cela accélère l'exécution en supprimant les opérations inutiles.
4. **Garbage Collector (Gestion de mémoire)** : V8 possède un Garbage Collector avancé qui libère automatiquement la mémoire des objets inutilisés.



# Exemple de code JavaScript et son exécution dans V8

```
function sum(a, b) {  
  return a + b;  
}  
console.log(sum(2, 3));
```

V8 lit le **code source** et le transforme en **Abstract Syntax Tree (AST)**, une représentation arborescente du code.

```
{  
  "type": "FunctionDeclaration",  
  "name": "sum",  
  "params": ["a", "b"],  
  "body": {  
    "type": "ReturnStatement",  
    "expression": {  
      "type": "BinaryExpression",  
      "operator": "+",  
      "left": "a",  
      "right": "b"}}}  
}
```

- Si la fonction `sum(2, 3)` est appelée une seule fois, elle restera en bytecode.
- Ignition, l'interpréteur de V8, prend l'AST et le convertit en bytecode.

```
mov eax, [rbp-8] // Charge `a` dans un registre  
add eax, [rbp-12] // Ajoute `b`  
ret // Retourne le résultat
```

- Si elle est appelée souvent, V8 détecte qu'elle est "chaude" (hot code) et l'envoie à TurboFan.

L'optimisation inclut des techniques comme **inlining** (remplacement d'un appel de fonction par son corps directement).

```
LdaNamedProperty a  
LdaNamedProperty b  
Add  
Return
```

# Les bases du JavaScript : JavaScript interne

**JavaScript interne:** Le JavaScript peut être inséré directement dans un document HTML à l'aide de la balise `<script>`. il est généralement inséré soit dans la section `<head>`, soit dans `<body>` du document.

```
<!DOCTYPE html>
<html>
<head></head>
<body>
<h2 style="color: blue;">
JavaScript interne : Code JS placé dans le code HTML
</h2>
<h3 id="demo" style="color: green;"> regardez la console </h3>

<script>
  // Ceci est un commentaire sur une seule ligne
  /* Ceci est un commentaire
    sur plusieurs lignes */

  console.log("Bonjour ILSI"); // Affiche un message dans la console du navigateur

</script>

</body>
</html>
```

# Les bases du JavaScript : JavaScript externe

**External JavaScript :** Pour les projets de grande envergure ou lorsque vous réutilisez des scripts sur plusieurs fichiers HTML, vous pouvez placer votre code JavaScript dans un fichier externe avec l'extension .js. Ce fichier est ensuite lié à votre document HTML en utilisant l'attribut src à l'intérieur d'une balise <script>.

## Avantages :

- **Temps de chargement plus rapides :** Les fichiers JavaScript externes sont mis en cache par le navigateur, ce qui évite de les recharger à chaque visite.
- **Meilleure lisibilité et maintenance :** Séparer le HTML et le JavaScript rend les deux plus faciles à lire et à maintenir.
- **Séparation des responsabilités :** En isolant la structure (HTML) du comportement (JavaScript), le code devient plus propre, mieux organisé et plus modulaire.
- **Réutilisation du code :** Un seul fichier JavaScript externe peut être utilisé avec plusieurs pages HTML. Cela réduit les duplications et simplifie les mises à jour.

```
// fichier index.html
<!DOCTYPE html>
<html>
<head></head>
<body>
<h2>JavaScript externe : Code JS dans un fichier séparé
</h2>
<h3 id="demo" style="color: green;"> regardez la console
</h3>
<!-- Lien vers le fichier JavaScript externe -->
<script src="demo1.js"></script>
</body>
</html>
```

```
// fichier demo1.js

console.log("Bonjour ILSI");
```

# Les bases du JavaScript : Versions de JavaScript (ECMAScript)

Aperçu des principales versions d'ECMAScript, de leurs années de sortie et des fonctionnalités clés qu'elles ont introduites.

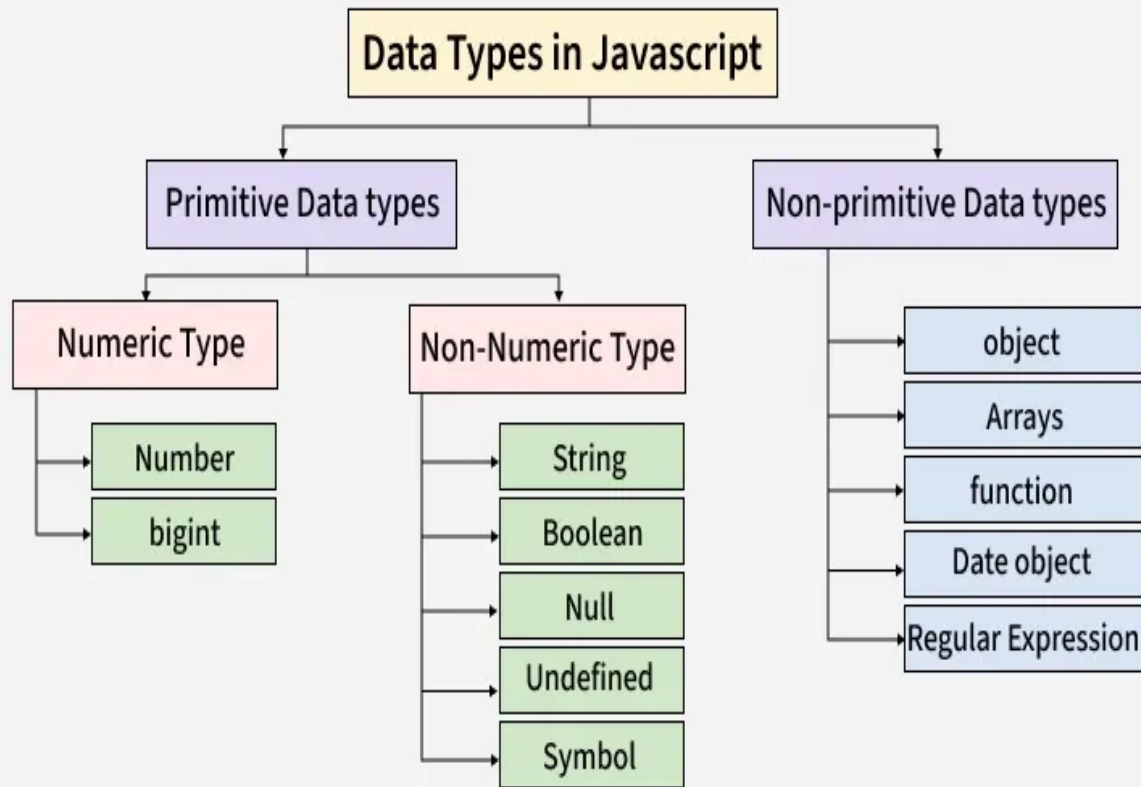
| Version    | Année       | Fonctionnalités clés                                         |
|------------|-------------|--------------------------------------------------------------|
| ES5        | 2009        | mode strict, prise en charge de JSON, getters/setters        |
| ES6        | 2015        | let / const, classes, fonctions fléchées                     |
| ES7 à ES13 | 2016 – 2022 | async/await, BigInt, opérateur d'enchaînement optionnel (?.) |
| ES14       | 2023        | toSorted, findLast, blocs statiques                          |

# Les bases du JavaScript : Data Types

JavaScript propose plusieurs types de données, que l'on peut regrouper en deux grandes catégories : les types primitifs et les types non primitifs.

Un type de données répond à ces questions :

- **Nature** : C'est quoi ? (Texte, nombre, booléen...)
- **Stockage** : Comment c'est mémorisé ? (Pile vs Tas, octets...)
- **Usage** : Qu'est-ce qu'on peut en faire ? (Calculs, méthodes, accès...)



# Les bases du JavaScript : Data Types

Les types de données **primitifs** représentent des valeurs simples et ne peuvent pas être modifiés (immuables).

**1. Number** : représente les valeurs numériques (entiers et décimales)

```
let n = 42; let pi = 3.14;
```

**2. String** : représente du texte entouré de guillemets simples ou doubles

```
let s = "Bonjour tout le monde !";
```

**3. Boolean** : représente une valeur logique (true ou false)

```
let bool = true;
```

**4. Undefined** : Variable déclarée mais sans valeur assignée.

```
let a; console.log(a); // affiche undefined
```

**5. Null** — Valeur intentionnellement vide

```
let data = null;
```

# Les bases du JavaScript : Data Types

**Types non primitifs en JavaScript :** Les types non primitifs sont des objets. Ils permettent de stocker des collections de données ou des entités plus complexes.

**Object** — Représente des paires clé-valeur : Un objet permet de regrouper plusieurs informations sous une même structure.

```
let obj = {  
  name: "ALI",  
  age: 25  
};
```

**Array** — Représente une liste de valeurs

```
let a = ["red", "green", "blue"];
```

**Function** — Représente un bloc de code réutilisable

```
function fun() {  
  console.log("ILISI 2025");  
}
```

# Les bases du JavaScript : Les variables

Une variable est un espace de stockage utilisé pour conserver en mémoire différents types de données. Elle peut ensuite être exploitée dans les scripts. Le nom d'une variable peut inclure uniquement des caractères alphanumériques, le symbole \$ (**dollar**) et le caractère \_ (**underscore**). Cependant, elle ne doit pas commencer par un chiffre ni porter le nom d'une fonction existante en JavaScript. Pour créer une variable, on la déclare et on lui attribue une valeur :

En JavaScript, les variables sont déclarées en utilisant les mots-clés **var**, **let** ou **const**.

- **Var** : Le mot-clé var est utilisé pour déclarer une variable. Il a une portée globale ou une portée fonctionnelle.

```
var n = 5;  
console.log(n);  
var n = 20; // réaffectation autorisée  
console.log(n);
```

- **let** : Le mot-clé let introduit dans ES6, a une portée de bloc et ne peut pas être redéclaré dans la même portée.

```
let n = 10;  
n = 20; // La valeur peut être mise à jour  
// let n = 15; // Ne peut pas être redéclarée dans  
// la même portée  
console.log(n);
```

- **const** : Le mot-clé const déclare des variables qui ne peuvent pas être réaffectées. Il a également une portée de bloc.

```
const n = 100;  
// n = 200; Ceci provoquera une erreur  
console.log(n);
```

# Les bases du JavaScript : Types de données

JavaScript prend en charge divers types de données, qui peuvent être largement classés en types primitifs et non primitifs:

Une variable peut être de type numérique, mais aussi une chaîne de caractères :

```
var text = 'J\'écris mon texte ici'; /* Avec des apostrophes, le \ sert à
échapper une apostrophe intégrée dans le texte, pour ne pas que Javascript
pense que le texte s'arrête là.*/
```

Une variable peut enfin être de type booléen (boolean), avec deux états possibles : vrai ou faux (true ou false).

## Les opérateurs arithmétiques :

On peut utiliser 5 opérateurs arithmétiques : l'addition (+), la soustraction (-), la multiplication (\*), la division (/) et le modulo (%). Le modulo est le reste d'une division. Par exemple : 10 : 3 qui équivaut à :

# Valeur vs Référence (Types primitifs vs Objets)

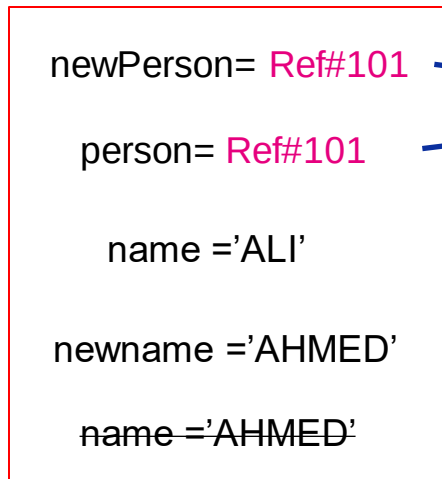
```
// ---EXEMPLE DES PRIMITIFS (copie par valeur) ----
```

```
let name = 'AHMED';  
let newName = name; // copie de la valeur 'Mike'  
name = 'ALI'; // modification de la variable originale  
// name vaut maintenant "AHMED"  
// newName vaut toujours "ALI"
```

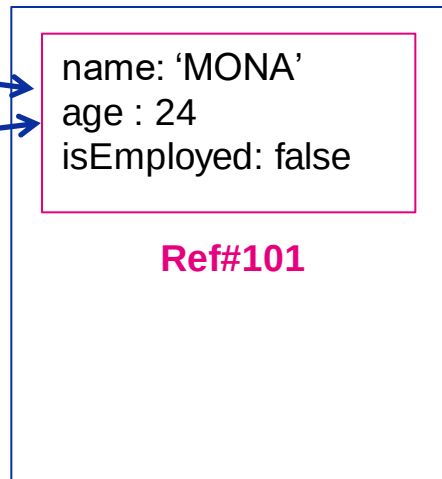
```
// ----- EXEMPLE DES OBJETS (copie par référence) -----
```

```
let person = {  
  name: 'MONA',  
  age : 24,  
  isEmployed : false  
};  
let newPerson = person; // copie de la référence ( pas duplication d'objet)  
person.name = 'ASMAA'; // modification de l'objet en mémoire  
// newPerson.name vaut aussi "ASMAA"
```

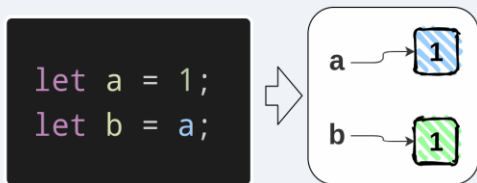
Stack



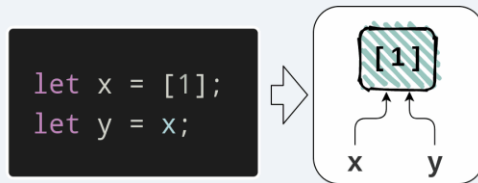
Heap



Values



References



# Les bases du JavaScript : Les opérateurs arithmétiques

## - Les opérateurs arithmétiques :

On peut utiliser 5 opérateurs arithmétiques : l'addition (+), la soustraction (-), la multiplication (\*), la division (/) et le modulo (%). Le modulo est le reste d'une division. Par exemple : 10 : 3 qui équivaut à :

```
var number1 = 3,  
    number2 = 2, result;  
result = number1 * number2;  
alert(result);  
//Affiche : « 6 »
```

```
var number = 3;  
number = number + 5;  
alert(number);  
//Affiche : « 8 »
```

```
var number = 3;  
number += 5;  
alert(number);  
//Affiche : « 8 »
```

- **La concaténation** : Une concaténation consiste à ajouter une chaîne de caractères à la fin d'une autre, comme dans cet exemple :

```
var hi = 'Bonjour ', name = 'ILISI';  
var result = hi + name;  
alert(result);  
// Affiche : « Bonjour ILISI »
```

```
var text = 'Bonjour ';  
text += 'ilisi';  
alert(text);  
// Affiche « Bonjour ilisi ».
```

On peut convertir un nombre en chaîne de caractères avec l'astuce suivante :

```
var text, number1 = 4, number2 = 2;  
text = number1 + ' ' + number2; /* on ajoute une chaîne de caractères vide  
entre les deux nombres pour permettre une concaténation sans calcul */  
alert(text); // Affiche : « 42 »
```

# Les bases du JavaScript : Les opérateurs de comparaison

## Les huit opérateurs de comparaison

Il y en a 8 :

== : égal à

!= : différent de

=== : contenu et type de variable égal à

!== : contenu ou type de variable différent de

> supérieur à

>= supérieur ou égal à

< inférieur à

<= inférieur ou égal à

# Les bases du JavaScript : Data Types

Exploration des types de données et des variables en JavaScript : Comprendre les expressions courantes

```
console.log(null === undefined)
```



```
false
```

```
console.log(5 > 3 > 2)
```



```
false
```

```
console.log([ ] === [ ])
```



```
false
```

```
console.log("10" < "9")
```



```
true
```

```
console.log("ALI"/5);
```



```
NaN
```

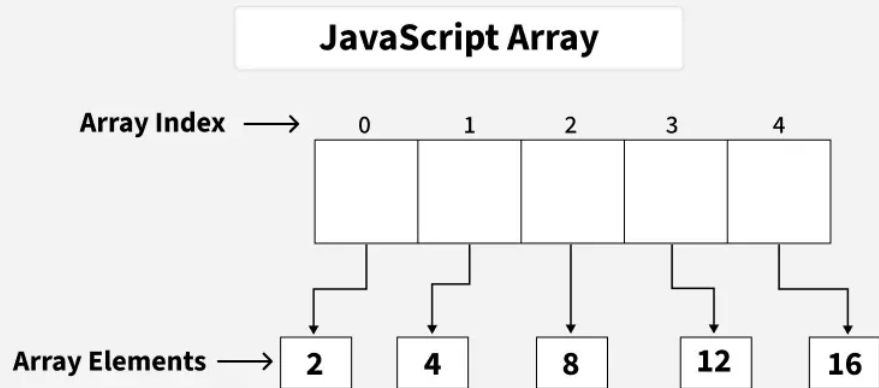
```
console.log(Infinity > 1000)
```



```
true
```

# Les bases du JavaScript : JavaScript Arrays

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Accessing Array Elements
console.log(a[0]); // first element
// Accessing Last Array Elements
console.log(a[a.length - 1]);
// Add Element to the end of Array
a.push("Node.js");
// Add Element to the beginning
a.unshift("Web Development");
console.log(a);
// Removes and returns the last element
let lst = a.pop();
console.log("After Removing the last: " + a);
// Removes and returns the first element
let fst = a.shift();
console.log("After Removing the First: " + a);
// Removes 2 elements starting from index 1
a.splice(1, 2);
console.log("After Removing 2 elements starting from index 1: " + a);
```



# Les bases du JavaScript : JavaScript Arrays



| Méthode    | Description                                                                | Exemple                               |
|------------|----------------------------------------------------------------------------|---------------------------------------|
| concat()   | Combine deux tableaux et retourne un nouveau tableau.                      | [1,2].concat([3,4]) // [1,2,3,4]      |
| filter()   | Retourne un tableau contenant uniquement les éléments qui passent un test. | [1,2,3,4].filter(n => n > 2) // [3,4] |
| pop()      | Supprime et retourne le dernier élément.                                   | let a=[1,2]; a.pop(); // 2            |
| slice()    | Retourne une partie du tableau sans le modifier.                           | [1,2,3].slice(0,2) // [1,2]           |
| unshift()  | Ajoute un ou plusieurs éléments au début.                                  | let a=[2,3]; a.unshift(1); // [1,2,3] |
| shift()    | Supprime et retourne le premier élément.                                   | let a=[1,2]; a.shift(); // 1          |
| sort()     | Trie un tableau (attention : tri alphabétique par défaut).                 | [3,10,1].sort() // [1,10,3]           |
| find()     | Retourne le premier élément qui respecte une condition.                    | [1,2,3].find(n => n > 1) // 2         |
| push()     | Ajoute un élément à la fin du tableau.                                     | let a=[1]; a.push(2); // [1,2]        |
| reverse()  | Inverse l'ordre du tableau.                                                | [1,2,3].reverse() // [3,2,1]          |
| map()      | Retourne un nouveau tableau après transformation de chaque élément.        | [1,2,3].map(n => n*2) // [2,4,6]      |
| splice()   | Modifie le tableau (ajout, suppression ou remplacement).                   | let a=[1,2,3]; a.splice(1,1) // [1,3] |
| join()     | Transforme un tableau en chaîne séparée par un séparateur.                 | ["a","b"].join("-") // "a-b"          |
| toString() | Transforme le tableau en texte (séparé par des virgules).                  | [1,2,3].toString() // "1,2,3"         |

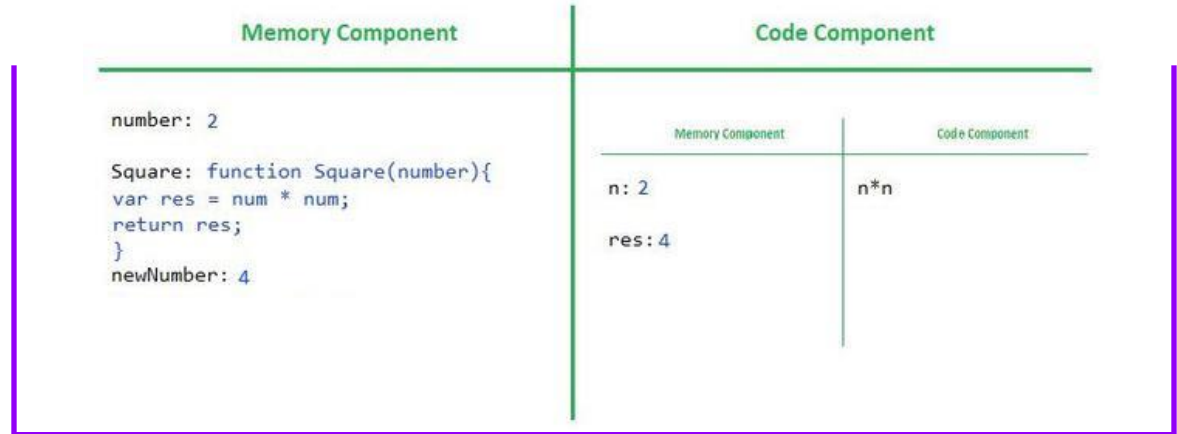
# Les bases du JavaScript : Le Contexte d'Exécution (Execution Context).

Tout code JavaScript s'exécute à l'intérieur d'un **contexte d'exécution**, qui est un concept abstrait servant de **conteneur**. Il contient toutes les informations sur **l'environnement d'exécution du code actuel**.

Lorsqu'un programme JavaScript s'exécute, un **contexte d'exécution est créé**, et son fonctionnement repose sur **deux phases principales**.

1. Phase d'Allocation Mémoire (Memory Allocation Phase) : JavaScript réserve de l'espace mémoire pour toutes les variables et fonctions du script.
2. Phase d'Exécution du Code (Thread d'exécution.) :

```
var number = 2;  
  
function square(num){  
  var res = num * num;  
  return res;  
}  
  
var square1 = square(number);
```



# Les bases du JavaScript : fonctions

## Fonctions déclarées (Function Declaration)

```
function changeLight(color) {  
    console.log(color);  
}
```

```
direBonjour(); // Fonction appelée avant sa  
définition
```

```
function direBonjour() {  
    console.log("Bonjour !");  
}
```

- Lorsqu'on a besoin d'une fonction réutilisable.
- Quand on veut bénéficier du **hoisting** (la fonction peut être appelée avant sa déclaration).

**Le hoisting** est un comportement en JavaScript où **les déclarations de variables et de fonctions sont déplacées en haut de leur portée** avant l'exécution du code. Cela signifie que vous pouvez utiliser une fonction ou une variable **avant même de la déclarer** dans votre code.

## Fonction anonyme (Function Expression)

```
const maFonction = function() {  
    console.log("Hello !");  
};
```

```
maFonction(); // Appelle la fonction
```

- Lorsqu'on veut assigner une fonction à une variable.
- Pour des callbacks (ex: setTimeout).

Contrairement aux fonctions déclarées, les fonctions anonymes **ne sont pas hoistées** !

# Les bases du JavaScript : fonctions

## Fonction en tant que paramètre (Callback Function)

Une **fonction callback** est une **fonction passée en argument** à une autre fonction et qui sera **exécutée plus tard**. Cela permet d'exécuter du code **asynchrone** ou de personnaliser le comportement d'une fonction.

```
function greet(name, callback) {  
    console.log("Hello, " + name);  
    callback();  
}  
  
function sayGoodbye() {  
    console.log("Goodbye!");  
}  
  
// Appel avec callback  
greet("Alice", sayGoodbye);
```

- Programmation événementielle (DOM)
- Flexibilité et réutilisabilité
- Couplage plus faible
- **Inversion de contrôle (IoC)**

## Fonctions fléchées (Arrow Functions)

```
const direBonjour = () =>  
console.log("Bonjour !");  
direBonjour(); // Affiche "Bonjour !"
```

- Les arrow functions (=>) sont une syntaxe plus courte pour écrire des fonctions en JavaScript. Elles ont été introduites avec ES6 (ECMAScript 2015) et présentent plusieurs avantages, **notamment une meilleure gestion du this**.

# Les bases du JavaScript : This

- **This** dans l'espace global

Dans l'espace global, la valeur de **this** est l'objet global.

Cet objet dépend de l'environnement d'exécution :

- Navigateur → window
- Node.js → global

- **This** dans une fonction normale

La valeur de **this** dépend de la manière dont la fonction est appelée.

=> En mode non strict :

si la fonction est appelée seule → **this** est remplacé par l'objet global.

=> En mode strict :

si la fonction est appelée seule → **this** vaut undefined.

# Les bases du JavaScript : This

- **This** dans les arrow functions
  - Les arrow functions n'ont pas leur propre this.
  - Elles héritent du **this** du **contexte lexical englobant**.
  - On parle de **lexical this**.
- **This** dans le DOM
  - Dans un gestionnaire d'événement avec une fonction normale :
  - **this** référence l'élément DOM qui a déclenché l'événement.
  - Avec une arrow function :
  - this n'est pas l'élément DOM (héritage lexical).

# Manipulation du DOM: DOM?

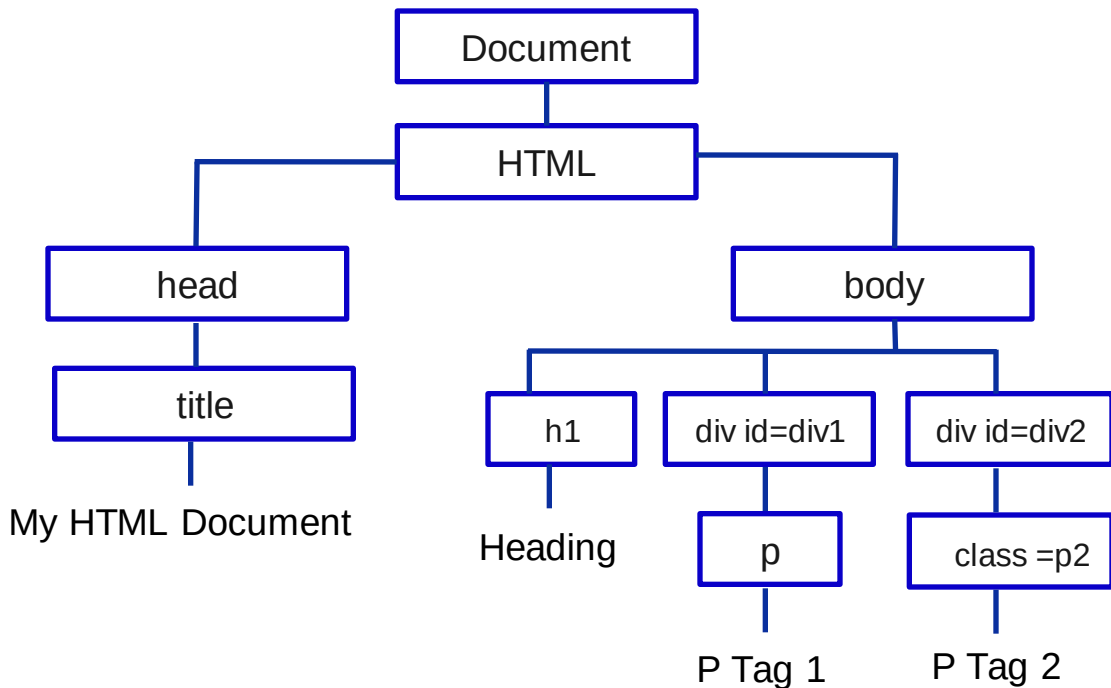
Le **DOM** (Document Object Model) est une représentation en mémoire de la page web sous forme d'un arbre d'objets.

**Chaque balise = un objet**

## HTML Document

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>My HTML Document</title>
5  </head>
6  <body>
7  |   <h1>Heading</h1>
8  |   <div id="div1">
9  | |   <p>P Tag 1</p>
10 | </div>
11 |   <div id="div2">
12 | |   <p class="p2">P Tag 2</p>
13 | </div>
14 </body>
15 </html>
```

## Document Object Model (DOM)



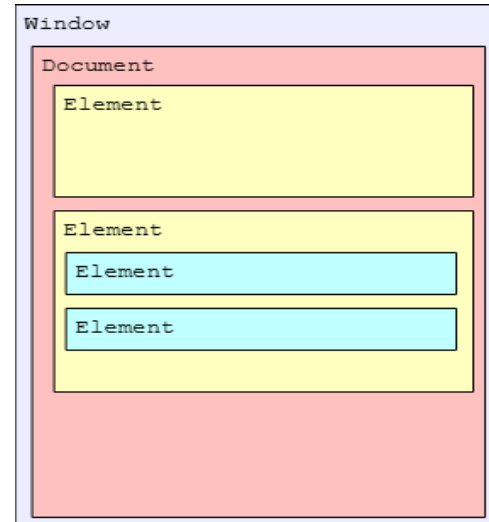
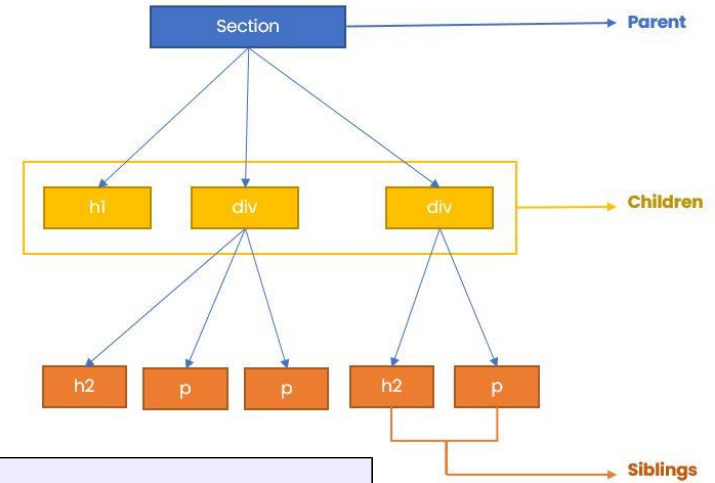
# Manipulation du DOM ?

## Pourquoi manipuler le DOM ?

La manipulation du DOM est essentielle pour rendre une page web dynamique et interactive. Sans cela, une page serait statique et l'utilisateur ne pourrait interagir avec elle.

Sans la manipulation du DOM, une page web resterait totalement statique :

- aucun changement de texte,
- aucun bouton interactif,
- aucun formulaire intelligent,
- aucune animation ni mise à jour automatique.



# Manipulation du DOM : DOM APIs?

Le **DOM API** est un ensemble d'interfaces JavaScript permettant d'interagir avec le **DOM** (Document Object Model).

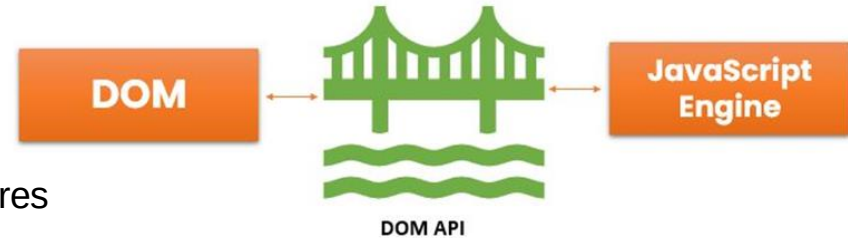
L'API DOM est indispensable pour rendre une page web interactive et dynamique. Elle fournit les méthodes nécessaires pour :

**C** – Create : Créer des éléments dans le DOM,

**R** – Read : Récupérer des éléments depuis le DOM

**U** – Update : Modifier des éléments existants,

**D** – Delete : Supprimer des éléments du DOM.



- Le **DOM** est ce que nous manipulons
- L'**API DOM** est ce qui nous permet de le manipuler.

# Manipulation du DOM : Récupération d'un Élément DOM

1. **document.getElementById("id")** : Sélectionne un élément grâce à son identifiant unique.

HTML

```
<h1 id="title">Bonjour</h1>
```

Js

```
const h1 = document.getElementById("title"); // sélection par id
```

2. **document.getElementsByClassName("class")** : Sélectionne tous les éléments possédant une classe donnée.

```
<p class="text">Paragraphe 1</p>  
<p class="text">Paragraphe 2</p>
```

```
const h1 = document.getElementsByClassName("text")
```

3. **document.getElementsByTagName("tag")** : Sélectionne tous les éléments ayant une balise (tag) donnée.

```
const paragraphs = document.getElementsByTagName("p");
```

# Manipulation du DOM : Récupération d'un Élément DOM

3. **document.querySelector(selector)** : Sélectionne le premier élément qui correspond au sélecteur CSS

## HTML

```
<h1 id="title">Bonjour</h1>
<p class="text">Paragraphe 1</p>
<p class="text">Paragraphe 2</p>
```

## Js

```
const h1 = document.querySelector("#title");    // sélection par id
const firstP = document.querySelector(".text"); // premier élément .text
const div = document.querySelector("div");      // première balise <div>
```

=> Gère tous les sélecteurs CSS : **.class**, **#id**, **tag**, **[attr]**, **etc.**

4. **document.querySelectorAll(selector)** : Sélectionne tous les éléments qui correspondent au sélecteur (NodeList).

```
const allParagraphs = document.querySelectorAll(".text");

allParagraphs.forEach(p => {
  console.log(p.textContent);
});
```

=> Indispensable pour modifier plusieurs éléments

# Manipulation du DOM : Récupération d'un Élément DOM

| Type de sélection | Exemple CSS        | querySelector()      |
|-------------------|--------------------|----------------------|
| ID                | #id                | "#box"               |
| Classe            | .class             | ".item"              |
| Balise            | div                | "div"                |
| Descendant        | ul li              | "ul li"              |
| Enfant direct     | div > p            | "div > p"            |
| Attribut          | input[type='text'] | "input[type='text']" |
| Pseudo-class      | li:first-child     | "li:first-child"     |
| Multiple          | h1, h2             | "h1, h2"             |

# Manipulation du DOM : Modification d'un Élément DOM

La modification du DOM consiste à changer le **contenu**, les **attributs**, les **classes**, ou le **style** d'un élément existant dans la page. JavaScript permet de modifier dynamiquement l'apparence et le comportement d'un élément sélectionné grâce aux propriétés du DOM.

1. Modifier le contenu texte : **textContent** => Change uniquement le texte

```
const title = document.querySelector("h1");  
title.textContent = "Nouveau titre";
```

2. Modifier le contenu HTML : **innerHTML** => Interprète du HTML

```
const div = document.querySelector(".box");  
div.innerHTML = "<p>Paragraphe <strong>ajouté</strong></p>";
```

3. Modifier les attributs : **setAttribute()**

```
const img = document.querySelector("img");  
img.setAttribute("src", "photo.png");  
img.setAttribute("alt", "Nouvelle image");
```

# Manipulation du DOM : Modification d'un Élément DOM

La modification du DOM consiste à changer le **contenu**, les **attributs**, les **classes**, ou le **style** d'un élément existant dans la page. JavaScript permet de modifier dynamiquement l'apparence et le comportement d'un élément sélectionné grâce aux propriétés du DOM.

## 4. Modifier les styles : **element.style**

```
title.style.color = "red";  
title.style.fontSize = "30px";
```

## 5. Modifier les classes : **classList**

```
title.classList.add("active"); // Add  
title.classList.remove("active"); // Remove  
title.classList.toggle("highlight"); // Alternner (toggle)  
title.classList.contains("active"); // Vérifier si une classe existe
```

# Manipulation du DOM : Création d'un Élément DOM

La création d'éléments dans le DOM se fait en 3 étapes :

1. Créer l'élément : **document.createElement("tag")**: Crée un nouvel élément HTML en mémoire

```
const div = document.createElement("div");
```

2. Modifier ses propriétés / contenu :

```
div.textContent = "Hello World!";  
div.style.color="purple";  
div.innerText= "Hello hello!";
```

3. **document.body.appendChild(node)** : Ajoute un nœud enfant à un élément existant dans le DOM.

```
document.body.appendChild(div);
```

# Manipulation du DOM : Suppression d'un Élément DOM

1. **element.remove()** : Supprime directement l'élément du DOM.

```
const p = document.querySelector("p");  
p.remove();
```

2. **parent.removeChild(child)** : Suppression via le parent

```
const list = document.querySelector("ul");  
const firstItem = list.firstElementChild;  
  
list.removeChild(firstItem);
```

3. **innerHTML = ""** : Vider complètement un élément => Permet de supprimer tout le contenu d'un élément.

```
const container = document.querySelector(".container");  
container.innerHTML = "";
```

# Manipulation du DOM : Gestion des événements dans le DOM

JavaScript nous permet de gérer des événements tels que les **clics**, les **frappes** au clavier, les **mouvements** de la souris, etc.

1. Ajouter un écouteur d'événement :

```
document.getElementById("btn").addEventListener("click", function() {  
    alert("Button Clicked!");  
});
```

2. En JavaScript, on utilise **removeEventListener()** pour supprimer un écouteur ajouté avec **addEventListener()**.

// 1. Définir une fonction séparée

```
function handleClick() {  
    console.log("Bouton cliqué !");  
}
```

// 2. Ajouter l'écouteur

```
btn.addEventListener("click", handleClick);
```

/ 3. Supprimer l'écouteur lorsqu'on clique sur "remove"

```
remove.addEventListener("click", () => {  
    btn.removeEventListener("click", handleClick);  
    console.log("Événement supprimé !");  
});
```

# Manipulation du DOM : Web APIs (setTimeout , setInterval)

- **setTimeout** est une Web API
  - Elle permet d'exécuter une fonction une seule fois, après un certain délai.

**setTimeout**(fonction, délai\_en\_millisecondes);

- **fonction** = le code à exécuter
- **délai\_en\_millisecondes**=temps d'attente (en ms)

```
setTimeout(() => {  
  
    console.log("Bonjour après 2 secondes");  
}, 2000);
```

=> Afficher Bonjour après 2 ms

# Manipulation du DOM : Web APIs (setTimeout , setInterval)

- **setInterval** est une Web API

Elle permet d'exécuter une fonction de manière répétée, avec un intervalle de temps fixe.

```
setInterval(fonction, délai_en_millisecondes);
```

- fonction = le code à exécuter
- délai\_en\_millisecondes=temps entre deux exécutions (en ms)

```
setInterval(() => {  
  
    console.log("Bonjour");  
  
}, 1000);
```

=> Afficher Bonjour chaque 1000 ms

# Manipulation du DOM : Web Storage API

Stockage côté client avec JavaScript

- JavaScript peut stocker des données côté client.

**Objectif** : garder des informations entre les pages ou pendant une session.

- Les Web Storage API sont fournies par le navigateur

| Web storage API | Durée de vie                                    | Exemple d'usage                               |
|-----------------|-------------------------------------------------|-----------------------------------------------|
| localStorage    | Persistant (même après fermeture du navigateur) | Stocker les préférences de l'utilisateur      |
| sessionStorage  | Session en cours (fermeture onglet → effacé)    | Stocker l'utilisateur connecté temporairement |
| Cookies         | Définie par l'expiration                        | Gestion de session serveur, suivi analytics   |

# Manipulation du DOM : Web Storage API

**API sessionStorage** : Stockage temporaire côté client, spécifique à un onglet.

**Syntaxe :**

```
sessionStorage.setItem("key", "value"); // Stocke une donnée
const value = sessionStorage.getItem("key"); // Récupère la donnée
sessionStorage.removeItem("key"); // Supprime une donnée
sessionStorage.clear(); // Vide tout
```

**Utilité** : transférer des données entre pages pendant une session.

**Exemple:**

```
// Stockage utilisateur après login
const user = { name: "Ahmed", balance: 1200 };
sessionStorage.setItem("currentUser", JSON.stringify(user));

// Récupération sur le dashboard
const currentUser = JSON.parse(sessionStorage.getItem("currentUser"));
console.log(currentUser.name); // "Ahmed"
```

# Manipulation du DOM : Web Storage API

**API localStorage:** Stockage permanent côté client.

**Syntaxe :**

```
localStorage.setItem("key", "value"); // Stocke une donnée
const value = localStorage.getItem("key"); // Récupère la donnée
localStorage.removeItem("key")          // Supprime une donnée
localStorage.clear();                   // Vide tout
```

**Utilité :** préférences, thèmes, tokens persistants.

**Exemple:**

```
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme"); // "dark"
```

# Manipulation du DOM : Web Storage API

**Les Cookies:** Stockage côté client pour sessions et suivi. Petit fichier texte stocké dans le navigateur.

Peut contenir :

- Identifiant de session
- Préférences utilisateur
- Données de suivi (analytics)

Envoyé automatiquement au serveur à chaque requête HTTP.

**Syntaxe :**

```
// Création d'un cookie
document.cookie = "username=Ahmed; max-age=3600; path="/";

// Suppression du cookie
document.cookie = "username=; max-age=0; path="/";
```

**Utilité :**

- Suivi des visites, pages consultées, interactions avec le site.
- Personnaliser les annonces en fonction des intérêts et du comportement de navigation.
- Stocker des choix de l'utilisateur : thème, langue, taille du texte, etc.

# Les bases du JavaScript : JS object vs Json Object

Un objet JavaScript est une structure de données qui suit la notation key: value, où les clés ne sont pas nécessairement des chaînes et les valeurs peuvent être de tout type (fonctions, objets imbriqués, undefined, etc.).

```
const user = {  
  name: "Alice",  
  age: 25,  
  isAdmin: false,  
  greet: function() {  
    console.log("Hello, " + this.name);  
  }  
};  
  
console.log(user.name); // Alice  
user.greet(); // Hello, Alice
```

```
{  
  "name": "Alice",  
  "age": 25,  
  "isAdmin": false  
}
```

## Objet JSON

Un objet JSON (JavaScript Object Notation) est une chaîne de texte qui suit un format spécifique, utilisé pour échanger des données entre serveurs et clients.

# Les bases du JavaScript : JS object vs Json

## Conversion entre Objet JavaScript et JSON

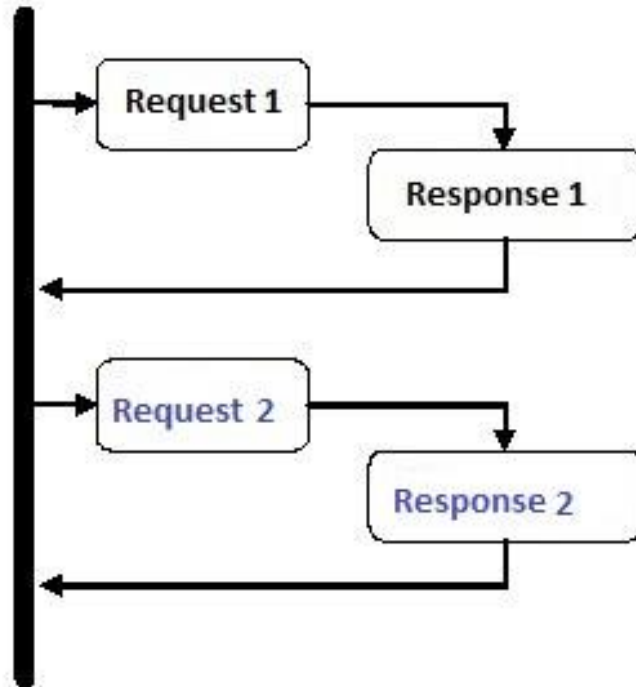
```
const user = {  
  name: "Alice",  
  age: 25,  
  isAdmin: false  
};  
  
const jsonString = JSON.stringify(user);  
console.log(jsonString);  
//  
{"name":"Alice","age":25,"isAdmin":false}
```

```
const jsonString = '{"name":  
"Alice", "age": 25, "isAdmin":  
false}';  
const userObject =  
JSON.parse(jsonString);  
  
console.log(userObject.name); //  
Alice  
console.log(userObject.age); // 25
```

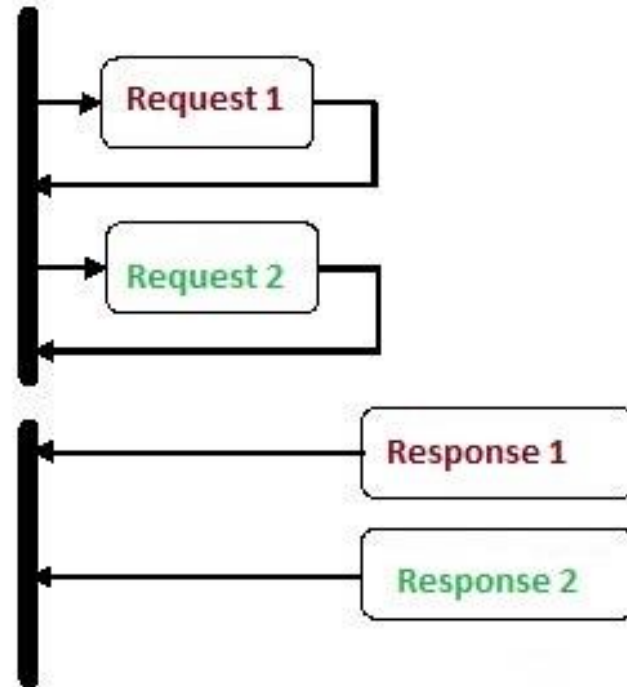
- Un **objet JavaScript** est utilisé en mémoire pour manipuler des données.
- Un **objet JSON** est une **chaîne** au format texte, utilisée pour le stockage et le transfert de données.

# Synchrone vs Asynchrone code

Synchronous



Asynchronous



# Synchrone Vs Asynchrone

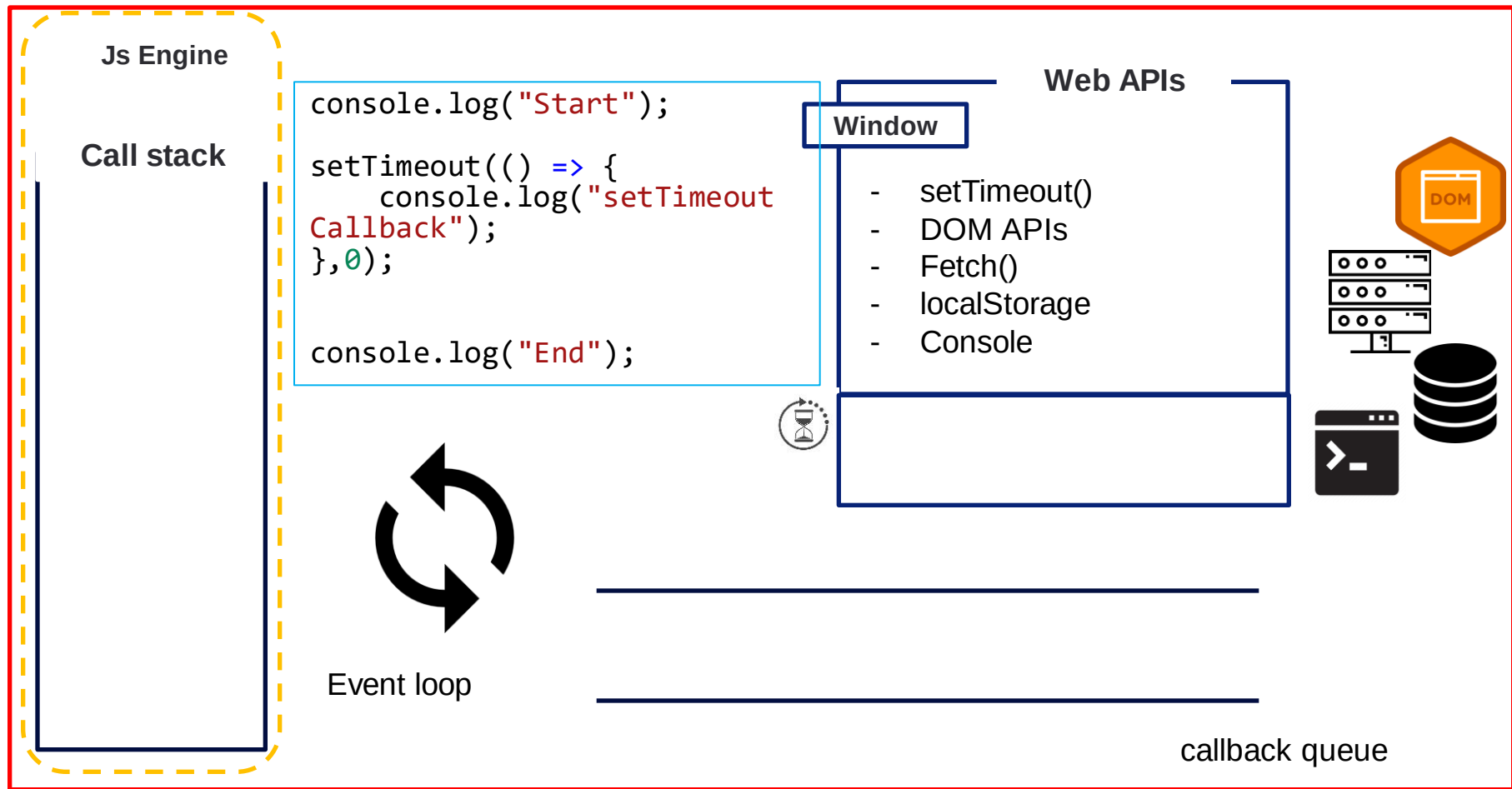


- Architecture bloquante
- L'exécution de chaque opération dépend de la fin de celle qui la précède



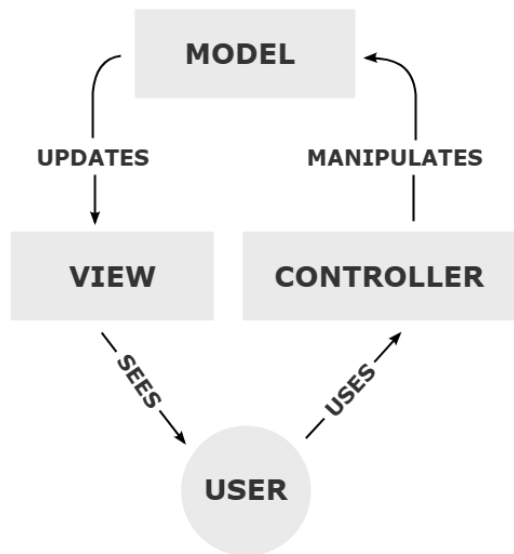
- Architecture non-bloquante
- L'exécution d'une tâche n'est pas dépendante d'une autre

# Les bases du JavaScript : Event loop



# Atelier (Manipulation DOM et Asynchrone) : MVC Pattern

MVC est un pattern architectural qui organise le code en **trois** parties distinctes



## Model (Modèle)

- Représente les données et la logique métier.
- Exemple : les utilisateurs, les produits,...etc.

## View (Vue)

- Représente l’affichage à l’utilisateur.
- Exemple : HTML + CSS pour le login et le dashboard.

## Controller (Contrôleur)

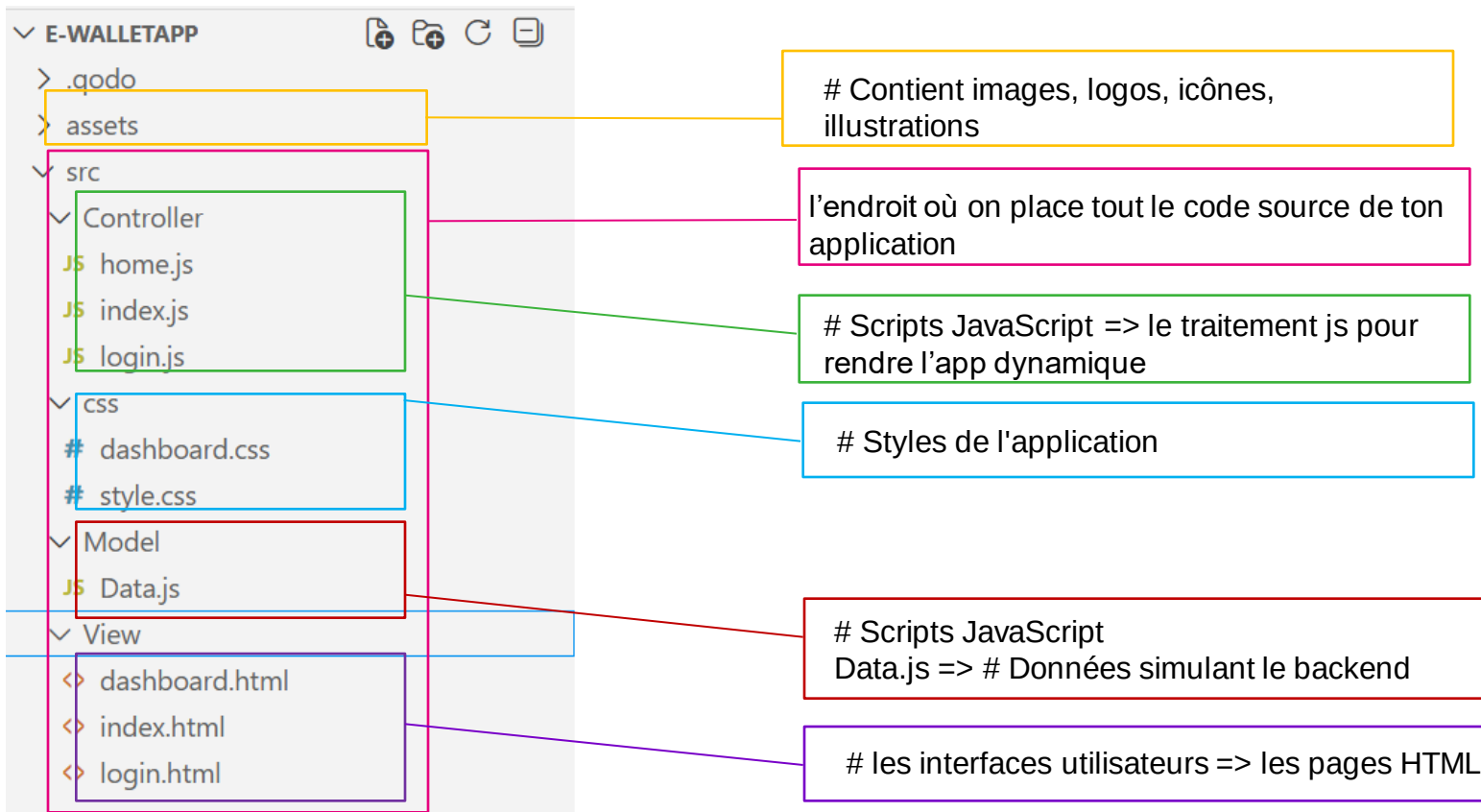
- Gère les interactions utilisateur et fait le lien entre Model et View.
- Exemple : gérer le clic sur le bouton login, vérifier l’utilisateur et mettre à jour l’affichage..

## Avantage :

- **Séparation des responsabilités** : chaque partie a un rôle clair .
- **Réutilisable** : on peut appliquer MVC à n’importe quelle application web ou desktop.
- **Facilite le débogage et l’évolution** : un changement dans la View n’impacte pas le Model.
- **Prépare aux frameworks modernes** : React, Angular, Vue, Django utilisent des concepts similaires.

# Atelier (Manipulation DOM et Asynchrone) : MVC Pattern

Etape1 : créer une arborescence claire, afin de séparer les rôles (HTML, CSS, JS, données).



## Atelier (E-wallet )

- Étapes d'un transfert bancaire
  - Vérifier que le client existe
  - Vérifier que le solde est suffisant
  - Mettre à jour le solde
  - Enregistrer la transaction
  - Mettre à jour l'interface
- Chaque étape dépend de la précédente
- Et chaque étape peut prendre du temps (appel serveur)

# Les bases du JavaScript : Notions Asynchrones

## Le problème de gestion des callbacks imbriqués

```
getUser(userId, (user) => {  
  getOrders(user, (orders) => {  
    processOrders(orders, (processed) => {  
      sendEmail(processed, (confirmation) => {  
        console.log("Order  
Processed:", confirmation);  
      });  
    });  
  });  
});
```

- Lorsqu'on utilise des callbacks pour gérer des opérations asynchrones (Ex: setTimeout, des requêtes réseau ou des accès DB), on peut rencontrer ce qu'on appelle le « callback hell » ou « pyramid of doom ».
- Ce problème apparaît lorsque plusieurs callbacks sont imbriqués les uns dans les autres, rendant le code difficile à lire, à maintenir et à déboguer.
- Il survient principalement dans des scénarios où chaque opération asynchrone dépend du résultat de la précédente, ce qui entraîne une structure de code en forme de pyramide.

- **Lisibilité réduite** : Le code devient difficile à lire et à comprendre avec l'augmentation des callbacks imbriqués.
- **Gestion des erreurs complexe** : Chaque callback doit vérifier s'il y a une erreur, ce qui peut être lourd, répétitif et difficile à suivre.
- **Maintenance compliquée** : Ajouter de nouvelles étapes ou modifier des étapes existantes dans une chaîne de callbacks peut entraîner des erreurs ou des incohérences.

**Le callback hell n'est pas un bug, c'est une mauvaise structure de code.**

# Les bases du JavaScript : Notions Asynchrones

## Le problème L'Inversion de Contrôle dans les Callbacks

L'inversion de contrôle, dans le contexte des callbacks, signifie que le développeur ne maîtrise plus directement le moment d'exécution de son code.

En passant une fonction callback, l'exécution est confiée à l'Event Loop, ce qui peut rendre le flux du programme plus difficile à comprendre, surtout lorsque les callbacks deviennent imbriqués.

```
console.log("A"); // exécuté maintenant  
console.log("B"); // juste après  
console.log("C"); // juste après
```

```
console.log("A");  
✓ setTimeout(() => {  
  |   console.log("B");  
  }, 1000);  
  
console.log("C");
```

- Dans le code synchrone, le “quand” est immédiat et prévisible.
- Dans le code asynchrone, le “quand” est délégué au moteur JavaScript.

# Les bases du JavaScript : Notions Asynchrones

## Le problème L'Inversion de Contrôle dans les Callbacks

L'inversion de contrôle, dans le contexte des callbacks, fait référence au fait que vous perdez une partie du contrôle sur l'exécution de votre code lorsque vous passez une fonction **callback** à une autre fonction. La fonction d'origine (celle qui appelle le callback) ne sait pas exactement quand la fonction de rappel sera exécutée, ni dans quel ordre. Cela crée une forme de **délégation de contrôle** qui peut entraîner une complexité croissante à mesure que le code devient plus imbriqué.

```
// Fonction principale
function createCommande() {
  console.log('Début de la commande...');

  getDataFromServer(function(data) {
    processData(data);
    saveData(data);
  });

  console.log('Commande en cours...');
}

createCommande();
```

Le problème principal ici est que la fonction **createCommande** délègue le contrôle à la fonction de rappel dans **getDataFromServer**, et elle ne peut pas contrôler l'ordre d'exécution des actions suivantes. Cela devient problématique quand :

- Il y a beaucoup de callbacks imbriqués.
- Le flux du programme devient difficile à suivre.
- Vous devez faire des appels multiples dans un ordre précis, et les erreurs dans l'une des étapes peuvent entraîner des effets en chaîne.

# Les bases du JavaScript : Notions Asynchrones (promise)

Une **Promise** en JavaScript est un objet représentant une **opération asynchrone** (qui prend du temps) qui **n'est pas encore terminée**, mais qui finira bien par donner un résultat dans le futur, ou échouer.

```
// creation de la promesse
let verifierSiPair =(nombre) =>{
  return new Promise((resolve, reject) => {
    if (nombre % 2 === 0) {
      resolve(`Le nombre ${nombre} est pair.`);
    } else {
      reject(`Le nombre ${nombre} est impair.`);
    }
  });
}

// Utilisation de la promesse
verifierSiPair(8)
  .then(message => console.log(message)) // succès (résolu)
  .catch(erreur => console.log(erreur)); //échec (rejeté)
```

Les promesses permettent d'enchaîner plusieurs opérations asynchrones de manière plus lisible et gérable, tout en centralisant la gestion des erreurs.

Une **Promise** peut être dans trois états différents :

1. **En attente (Pending)** : La promesse n'est pas encore terminée. L'opération asynchrone est en cours.
2. **Réussie (Fulfilled)** : L'opération a été terminée avec succès, et tu as le résultat que tu attendais.
3. **Échouée (Rejected)** : L'opération a échoué, et il y a une erreur ou un problème.\*

# Les bases du JavaScript : Notions Asynchrones (Promise)

Les promesses permettent de mieux gérer les opérations asynchrones en offrant une syntaxe plus claire et en réduisant les risques de "callback hell".

```
function fetchData(url) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = true; // Simulons un succès
      if (success) {
        resolve("Données de l'API");
      } else {
        reject("Erreur lors de la récupération des données");
      }
    }, 2000);
  });
}

fetchData("https://exemple.com")
  .then((data) => {
    console.log("Réponse : ", data);
  })
  .return getData("https://exemple2.com"); // Retourne une nouvelle promesse
  .catch((error) => {
    console.log("Erreur : ", error);
  });
```

- **Chaining (Chaînage)** : Les promesses permettent d'enchaîner plusieurs opérations asynchrones de manière plus lisible, sans avoir à imbriquer les fonctions callback.
- **Gestion des erreurs améliorée** : Avec les promesses, les erreurs sont capturées de manière centralisée via **.catch()**, ce qui rend la gestion des erreurs plus simple et plus lisible.
- **Syntaxe plus claire** : Moins de risque de "callback hell" avec le chaînage des promesses. Cela rend votre code plus lisible et plus facile à maintenir.
- **Gestion asynchrone plus fluide** : Une promesse peut être en attente (**pending**), résolue (**fulfilled**) ou rejetée (**rejected**), ce qui permet de mieux gérer les flux d'exécution.

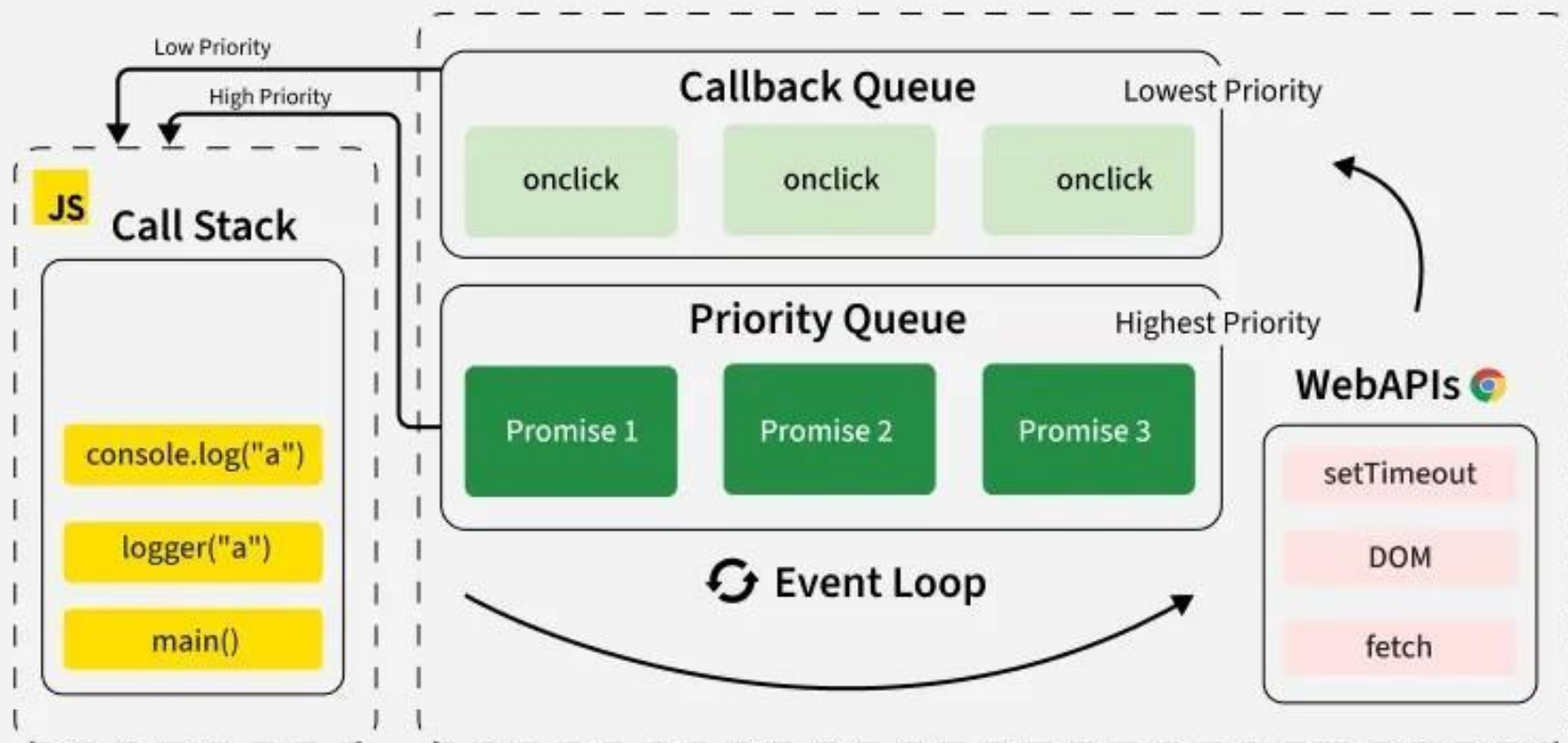
# Les bases du JavaScript : Notions Asynchrones (Promise)

## méthodes `.then()` et `.catch()` :

- **`.then()`** : Cette méthode est utilisée pour gérer la résolution d'une Promise. Elle prend une fonction callback qui sera exécutée lorsque la Promise sera résolue avec succès (c'est-à-dire, lorsque l'état de la Promise passe de en attente à résolue). La valeur que la Promise renvoie à l'intérieur de la fonction `resolve()` sera passée en paramètre à la fonction de rappel dans `.then()`.
- **`.catch()`** : Cette méthode est utilisée pour gérer le rejet d'une Promise. Elle prend une fonction callback qui sera exécutée lorsque la Promise sera rejetée. La raison du rejet (en général un objet d'erreur) sera passée en paramètre à la fonction de callback dans `.catch()`.

```
let checkEven = new Promise((resolve, reject) => {  
  let number = 4;  
  if (number % 2 === 0)  
    resolve("The number is even!");  
  else  
    reject("The number is odd!");  
});  
checkEven  
  .then((message) => console.log(message)) // On success  
  .catch((error) => console.error(error)); // On failure
```

# Les bases du JavaScript : promises



# Les bases du JavaScript : Notions Asynchrones (async /await)

**Async** : est utilisé pour déclarer une fonction asynchrone. Une fonction déclarée avec **async** retourne toujours une **Promise**.

```
async function example() {  
  return 42;  
}
```



```
function example() {  
  return Promise.resolve(42);  
}
```

Si la fonction retourne une valeur qui n'est pas une Promise, JavaScript la **convertit automatiquement en une Promise résolue**.

Les erreurs levées à l'intérieur d'une fonction **async** entraînent le **rejet de la Promise retournée**.

# Les bases du JavaScript : Notions Asynchrones (async /await)

**async / await** est une syntaxe moderne de JavaScript qui permet d'écrire du code asynchrone de manière plus lisible, plus proche du code synchrone.

- **async / await** repose entièrement sur les Promises
- Ce n'est pas un nouveau mécanisme, **seulement une nouvelle écriture**
- **async / await** ne change PAS la création de la Promise=> Il change uniquement la façon de l'utiliser.

```
let checkEven = new Promise((resolve, reject) => {  
  let number = 4;  
  if (number % 2 === 0)  
    resolve("The number is even!");  
  else  
    reject("The number is odd!");  
});
```

```
checkEven  
  .then((message) => console.log(message))//succes  
  .catch((error) => console.error(error));//echec
```

```
async function testerNombre() {  
  const message = await checkEven;  
  console.log(message); // succès  
}  
testerNombre();
```

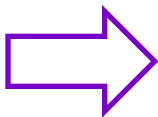
# Les bases du JavaScript : Responsabilité Unique (SRP)

La Solution = l'utilisation des **services** pour implementer le principe de Responsabilité Unique (SRP).

**Service** : Un service est une unité logique indépendante du UI. Il permet de centraliser les actions techniques (API, Storage, logique métier) pour rendre le code réutilisable et plus facile à tester.

En utilisant des services, nous transformons une structure désordonnée en une architecture professionnelle inspirée de frameworks comme NestJS

```
src > Controller > JS dashboard.js > ...
1  let user = JSON.parse(sessionStorage.getItem("user"));
2  const welcome_message = document.getElementById("welcome_message");
3  const balance = document.getElementById("balance");
4  const transactionsBody = document.querySelector("#transactions tbody");
5  const filterType = document.getElementById("filterType");
6  const transfertbtn=document.getElementById("transferrer");
7  welcome_message.textContent = `Bienvenue ${user.name}`;
8  balance.textContent = `${user.balance} MAD`;
9  transfertbtn.addEventListener("click",handletransfer);
10 | //AFFICHAGE TRANSACTIONS
   Qodo: Test this function
11 > function afficherTransactions(list) { ...
42 | }
43 | //FILTRAGE
44 |
45 > filterType.addEventListener("change", () => { ...
56 | });
57 | // AFFICHAGE INITIAL
58 afficherTransactions(user.transactions);|
   Qodo: Test this function
59 function handletransfer(){
60     setTimeout(()=>{
61         window.location.href="/src/View/Transfer.html"
62     },1000);
```



-> Gestion UI

-> la logique métier

# Les bases du JavaScript : Gestion des erreurs try/catch

Pourquoi la gestion des erreurs est essentielle ?

## Dans une application réelle :

- une requête peut échouer
- une donnée peut être invalide
- un service externe peut être indisponible
- une règle métier peut être violée

## Sans gestion des erreurs :

- l'application plante
- l'utilisateur ne comprend pas ce qui se passe
- le système devient difficile à maintenir

La gestion des erreurs permet de rendre l'application fiable, lisible et robuste.

# Les bases du JavaScript : Gestion des erreurs try/catch

Permettre au programme de continuer à s'exécuter même lorsqu'une erreur survient en interceptant et en gérant l'exception

## Bloc try

- Contient le code susceptible de générer une erreur
- Si une erreur se produit, l'exécution passe au bloc catch correspondant.

## Bloc catch

- Intercepte et gère l'exception
- Permet d'exécuter du code alternatif en cas d'erreur

## Bloc finally (Optionnel)

- Toujours exécuté, que l'erreur survienne ou non
- Idéal pour nettoyer des ressources (fermer fichiers, libérer mémoire ...)

## Flux d'exécution

- Le code du bloc try est exécuté
- Si pas d'erreur → catch ignoré
- Si erreur → exécution de catch
- finally s'exécute toujours (si présent)

# Les bases du JavaScript : Gestion des erreurs try/catch

## Syntaxe générale:

```
try {  
    // code à risque  
} catch (error) {  
    // gestion de l'erreur  
}  
finally {  
    // code exécuté dans tous les cas  
}
```

- On ne peut pas utiliser catch sans try
- On ne peut pas utiliser plusieurs try avec un seul catch partagé
- On peut avoir plusieurs catch pour gérer différents types d'erreurs
- Le finally est facultatif mais très utile pour la propreté du code

La structure est similaire dans de nombreux langages (C++,JS,Java, Python).

# Les bases du JavaScript : throw + try / catch

## Syntaxe générale:

```
async function payer(amount) {  
  if (amount <= 0) {  
    thrownew Error("Montant invalide");  
  }  
  return"Paielement réussi";  
}  
try {  
  await payer(-10);  
} catch (e) {  
  console.log(e.message);  
}
```

**throw** permet de lever (déclencher) une erreur volontairement.

On utilise **throw** lorsqu'une règle métier n'est pas respectée ou lorsqu'une situation est invalide, même si JavaScript ne génère pas automatiquement d'erreur.

**throw** permet de signaler explicitement une erreur et de forcer son traitement via **try / catch**.

# Pourquoi migrer vers React ?

## Les limites du JavaScript vanilla

### Couplage fort

- Le JavaScript doit connaître exactement les balises HTML (div) et les classes CSS (transaction-item)
- Si un designer modifie une classe CSS, il faut corriger le code JavaScript

```
transactionsList.innerHTML = "";
user.wallet.transactions.forEach(transaction => {
  const transactionItem = document.createElement("div");
  transactionItem.className = "transaction-item";
  transactionItem.innerHTML = `
    <div>${transaction.date}</div>
    <div>${transaction.amount} MAD</div>
    <div>${transaction.type}</div>
  `;
  transactionsList.appendChild(transactionItem);
});
```

### Impératif (“Comment”) vs Déclaratif (“Quoi”)

- En JavaScript vanilla, on donne des instructions détaillées :  
“Créer un div, ajouter une classe, insérer du contenu, puis l’ajouter au parent” (on décrit comment faire)
- Si on oublie : `transactionsList.innerHTML = ""`; => la liste continue de grandir indéfiniment =>

### Faible réutilisabilité

- Impossible de réutiliser facilement le code
- Nécessite de copier-coller toute la logique
- Difficulté à maintenir et à faire évoluer

**NB** : Copier-coller le code, c’est multiplier les problèmes.  
Réutiliser le code, c’est simplifier la maintenance

# Pourquoi migrer vers React ?

Les limites du JavaScript vanilla

## Manipulation directe du DOM

- Utilisation de **getElementById**, **querySelector**
- Mises à jour manuelles répétitives

## Code difficile à maintenir

- Logique métier mélangée avec le DOM
- Risque d'oublier des mises à jour
- Code rapidement désorganisé



**Solution**

## La transition vers l'approche déclarative

On arrête de « **construire** » l'interface étape par étape, et on commence à la « **décrire** ».

## Mauvaise gestion de l'état

- Synchronisation complexe entre données et interface
- Bugs fréquents (affichage incohérent)

## Difficulté à scaler

- Réutilisation limitée
- Pas de structure claire pour les grandes applications

# Pourquoi migrer vers React ?

## JavaScript vanilla (impératif — *comment faire*)

```
const transactions = ["+100 DH", "-50 DH"];

const list = document.getElementById("list");
list.innerHTML = "";

transactions.forEach((t) => {
  const li = document.createElement("li");
  li.textContent = t;
  list.appendChild(li);
});
```

Tu expliques **chaque étape** :

- créer un élément
- ajouter du contenu
- l'insérer dans le DOM

## React (déclaratif — *quoi afficher*)

```
function App() {
  const transactions = ["+100 DH", "-50 DH"];
  return (
    <ul>
      {transactions.map((t, index) => (
        <li key={index}>{t}</li>))}
    </ul>);
}
```

Tu **décris** simplement :

- afficher une liste basée sur les données

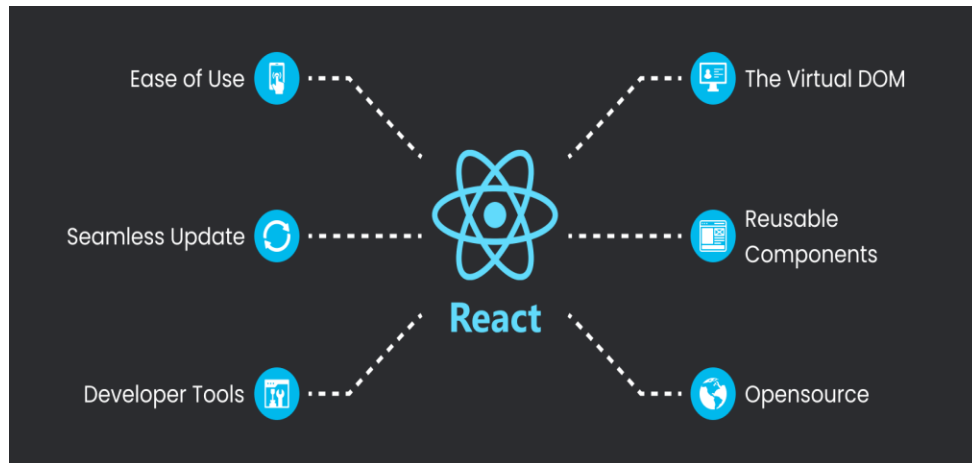
**NB** : En JavaScript, on construit l'interface étape par étape.  
En React, on décrit l'interface à partir des données.

# Introduction à React.js

**React.js** est une bibliothèque **JavaScript** populaire et **open-source** utilisée pour construire des interfaces utilisateur (UI), principalement pour des applications à page unique (SPA - **Single Page Applications**). Développée par Facebook (Meta), elle est maintenue par une large communauté de développeurs. React permet aux développeurs de créer des applications web rapides, évolutives et faciles à maintenir, avec un accent particulier sur le rendu des interfaces utilisateur.

## Principales caractéristiques de React.js :

- **Architecture basée sur des composants** : React utilise une architecture basée sur des composants, ce qui signifie que l'interface utilisateur est divisée en petites unités **réutilisables** appelées composants.
- Chaque composant contient sa propre **logique**, son **rendu** et son **état**. Cela facilite la gestion et l'évolution des applications.



```
function Greeting() {  
  return <h1>Bonjour, le monde !</h1>;  
}
```

# Introduction à React.js

```
document.getElementById("balance").textContent = balance;
```

```
<h2>Balance: {balance}</h2>
```

|                      |                         |
|----------------------|-------------------------|
| JavaScript vanilla   | React                   |
| Manipulation du DOM  | Manipulation du DOM     |
| Mise à jour manuelle | Mise à jour automatique |
| Approche impérative  | Approche déclarative    |

- Le développeur ne touche plus directement au DOM
- React sert d'intermédiaire intelligent
  - Les mises à jour sont :
    - automatiques
    - optimisées

**React** met à jour automatiquement l'interface quand les **données changent**.

# Configuration et Installation d'un Projet React

## Prérequis :

- Installer **Node.js**
- Vérifier l'installation avec les commandes suivantes :

```
node -v  
npm -v
```

**Vite** est l'outil qui nous permet de créer et exécuter facilement notre application React en local, avec des mises à jour instantanées.

## Création d'un projet React avec vite :

```
npm create vite@latest front-end -- --template react  
Cd front-end  
npm install  
npm run dev
```

## Structure du projet généré :

- **public/** : Contient index.html et les fichiers statiques.
- **src/** : Contient le code source React.
- **node\_modules/** : Bibliothèques installées via npm.
- **package.json** : Liste des dépendances et scripts de gestion du projet.

# ReactJS : Java Script XML (JSX)

**JSX** (JavaScript Syntax Extension et parfois appelé JavaScript XML) est une extension React de la syntaxe du langage JavaScript qui permet de structurer le rendu des composants à l'aide d'une syntaxe familière à de nombreux développeurs.

```
<h1>Liste de tâches</h1>
<img src =" url"      class="photo">
<ul>
<li>Préparer une présentation technique
<li>Rédiger un rapport de projet
<li>Planifier les étapes d'un sprint
</ul>
```

JSX

```
export default function TodoList() {
  return (
    // ???
  )
}
```

## Les règles de JSX

### 1. Renvoyer un seul élément racine

- Un composant React doit retourner un seul élément parent
- Pour regrouper plusieurs éléments :
  - utiliser une balise parent (**div**, etc.)
- Alternative (sans ajouter de balise inutile)
  - Utiliser un Fragment **<></>**

```
<>
<h1>Liste de tâches</h1>
<img src =" url"      class="photo">
<ul>
<li>Préparer une présentation technique
<li>Rédiger un rapport de projet
<li>Planifier les étapes d'un sprint
</ul>
</>
```

# ReactJS : Java Script XML (JSX)

## 2. Fermez toutes les balises

**JSX** exige que les balises soient explicitement fermées : les balises **auto-fermantes** telles que `<img>` doivent utiliser `<img />`, et les balises enrobantes telles que `<li> oranges` doivent utiliser `<li>oranges</li>`.

```
<>
<h1>Liste de tâches</h1>
<img src = " url"      class="photo" />
<ul>
  <li>Préparer une présentation technique </li>
  <li>Rédiger un rapport de projet </li>
  <li>Planifier les étapes d'un sprint </li>
</ul>
</>
```

## 3. Utilisez la casseCamel pour (presque) tout

- class → className
- for → htmlFor
- camelCase obligatoire
- style = objet

| HTML        | JSX             |
|-------------|-----------------|
| class       | className       |
| for         | htmlFor         |
| onclick     | onClick         |
| onchange    | onChange        |
| tabindex    | tabIndex        |
| readonly    | readOnly        |
| maxlength   | maxLength       |
| style="..." | style={{ ... }} |
| <img>       | <img />         |
| <input>     | <input />       |
| <!-- -->    | {/* */}         |

# ReactJS : Java Script XML (JSX)

## JavaScript dans JSX avec {}

Les accolades {} permettent d'insérer du JavaScript dans JSX

```
function App() {  
  const name = "Ahmed";  
  return (  
    <h1>Bonjour {name}</h1>  
  );  
}
```

## On peut utiliser :

- variables
- expressions
- fonctions

```
function App() {  
  const user={  
    name:"Ahmed"}  
  function getMessage(){  
    console.log("this is getMessage");}  
  return (  
    <>  
    <h2>{2 + 2}</h2>  
    <h2>{user.name}</h2>  
    <h2>{getMessage()}</h2>  
    </>  
  );  
};
```

## Attention

On ne met que des expressions, pas des instructions

```
{if (x > 0) {...}}
```

# React Component

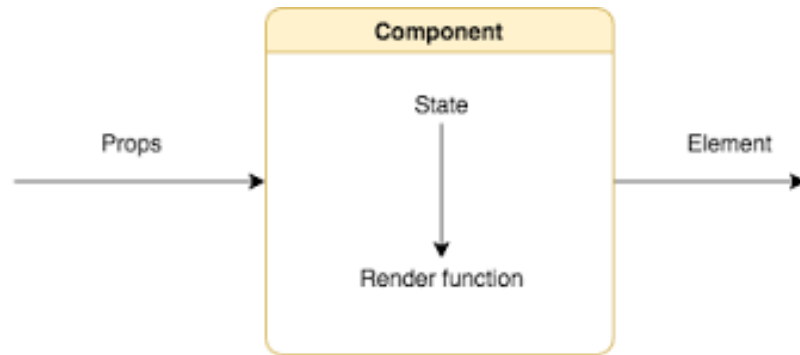
Un **composant React** est une **portion d'UI réutilisable et indépendante** qui peut gérer son propre état, recevoir des données (props) et décrire ce qui doit **s'afficher à l'écran**.

Un composant React est défini par son rendu, son état, ses propriétés et sa logique.

$$\text{UI} = F(\text{Props}, \text{Etat})$$

Un composant React fonctionne comme une **fonction**.

- **Entrée** → props
- **Mémoire interne** → state
- **Sortie** → JSX (UI)



- **Props** : données externes, en lecture seule, transmises par le parent.
- **État** : données internes, modifiables, propres au composant.

# ReactJS : Components et Props

Props = paramètres d'une fonction

- Les props viennent du **parent**
- Elles sont **en lecture seule (immutables)**
- Elles servent à **configurer le composant**

JSX props → JavaScript object

```
function Component(props) {  
  return <UI />;  
}
```

```
function Greeting({ name }) {  
  return <h1>Bonjour {name}</h1>;  
}  
  
function App() {  
  return <Greeting name="Ahmed" />;  
}
```

```
function User(props) {  
  props.name = "Ali"; // erreur  
}
```

```
<Component  
  name="Mouna"           // string  
  age={25}               // number  
  isLoggedIn={true}      // boolean  
  data={{ id: 1 }}       // object  
  list={[1, 2, 3]}       // array  
  onClick={() => {}}      // function  
>
```

# ReactJS : Components et Props

```
<Component name="Mouna" age={25} />
```

À l'intérieur du composant :

```
props = {  
  name: "Mouna",  
  age: 25  
}
```

Deux façons d'accéder aux props

Méthode 1 — en utilisant props

```
function Component(props) {  
  return <h1>{props.name}</h1>;  
}
```

Méthode 2 — en utilisant le **destructuring**

```
function Component({ name }) {  
  return <h1>{name}</h1>;  
}
```

Equivalent a

```
function Component(props) {  
  const {name}= props; // destructuring  
  return <h1>{name}</h1>;  
}
```

1. props est un objet
2. il regroupe toutes les données
3. il est envoyé du parent vers l'enfant
4. chaque composant reçoit toujours props
5. props peut être vide {}

# ReactJS : Components et States

Le **state** est un **objet** contenant des données **dynamiques** pouvant évoluer au fil du temps dans un composant React. Lorsque ces données changent, React met automatiquement à jour le rendu du composant

- Pas de modification directe du state
- Utiliser le setter pour toute mise à jour
- Prendre en compte l'état précédent si nécessaire
- State local à chaque composant
- Mises à jour non immédiates (asynchrones)

```
import React, { useState } from 'react';
function Counter() {
  // Déclaration du state
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Compteur: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Incrémenter
      </button>
    </div>
  );
}
```

# ReactJS : Le cycle de vie d'un composant

Le cycle de vie d'un composant React correspond aux **phases de création (montage)**, de **mise à jour (re-render)** et de **suppression (démontage)**, gérées avec **useEffect** dans les composants fonctionnels.

## 1. Montage (Mounting)

Le **montage** = le moment où le composant est **créé et affiché pour la première fois**.

1. React crée un Fiber node
2. React appelle la fonction du composant
3. JS crée un execution context
4. useState initialise le state
5. JSX est retourné
6. React crée le VDOM
7. React insère le DOM dans la page

```
export default function Component() {  
  const [count, setCount] = useState(0);  
  
  const handleCount = () => {  
    setCount(count + 1);  
    setCount(count + 1);  
    console.log(count);  
  };  
  return (  
    <div>  
      <h1>Compteur</h1>  
      <p>Valeur : {count}</p>  
      <button  
onClick={handleCount}>Incrémenter</button>  
    </div>  
  );  
}
```

```
const handleCount = () => {  
  setCount(prev => prev + 1);  
  setCount(prev => prev + 1);  
};
```

# ReactJS : Le cycle de vie d'un composant

Le cycle de vie correspond aux étapes : **montage** → **mise à jour** → **démontage**

## 2. Phase 2 — Mise à jour (Updating)

La phase **de mise à jour** correspond au moment où un composant est **re-render** suite à une modification (state, props, etc.).

### Déclencheurs

- changement de state
- changement de props
- re-render du parent

1. setState → enqueue update
2. React traite la queue
3. React appelle le composant
4. nouveau JSX
5. diffing (reconciliation) (ancien vDOM vs nouveau vDOM)
6. mise à jour du DOM

```
export default function Component() {  
  const [count, setCount] = useState(0);  
  
  const handleCount = () => {  
    setCount(count + 1);  
    setCount(count + 1);  
    console.log(count);  
  };  
  return (  
    <div>  
      <h1>Compteur</h1>  
      <p>Valeur : {count}</p>  
      <button  
        onClick={handleCount}>Incrémenter</button>  
    </div>  
  );  
};
```

```
const handleCount = () => {  
  setCount(prev => prev + 1);  
  setCount(prev => prev + 1);  
};
```

# ReactJS : Le cycle de vie d'un composant

Le cycle de vie correspond aux étapes : **montage** → **mise à jour** → **démontage**

## 3. Phase 3 — Démontage (Unmounting)

Le démontage est la phase où un composant est **retiré de l'interface et détruit logiquement par React**.

Un composant est supprimé quand :

- il disparaît du JSX (ex: condition)
- on change de page (routing)
- le parent ne le rend plus

1. React décide de supprimer le composant
2. exécution des fonctions de nettoyage (cleanup)
3. suppression du DOM
4. suppression du Fiber

```
export default function Component() {  
  const [count, setCount] = useState(0);  
  
  const handleCount = () => {  
    const [count, setCount] = useState(0);  
    setCount(count + 1);  
    setCount(count + 1);  
    console.log(count);  
  };  
  return (  
    <div>  
      <h1>Compteur</h1>  
      <p>Valeur : {count}</p>  
      <button  
        onClick={handleCount}>Incrémenter</button>  
    </div>  
  );  
}
```

{show && <Component />}

# Concepts de base de React Native : React Engine

Fiber = un arbre interne qui représente ton UI.

- Chaque composant = un Fiber Node
- C'est un objet JavaScript interne, créé et géré par React.

```
FiberNode {  
  type: Counter,  
  Hooks : [  
    Hook1 = {  
      memoizedState: 0,  
      queue: {  
        dispatch: setCount,  
        ...}  
      }  
    ]  
  child: fiber_button,  
  sibling: null,  
}
```



## React Lifecycle with Virtual DOM and Fiber

### Component Lifecycle

#### Mounting

constructor  
componentDidMount

#### Updating

shouldComponentUpdate  
componentDidUpdate

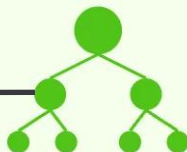
#### Unmounting

componentWillUnmount

#### Hooks

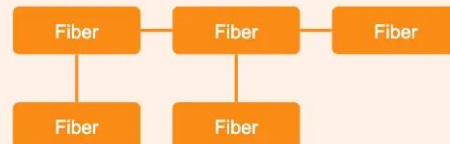
useState, useEffect  
useMemo, useCallback

### Virtual DOM



- JavaScript objects
- Lightweight DOM representation
- In-memory tree structure
- Used for efficient diffing
- Batch updates to real DOM

### Fiber Architecture



### React Reconciliation Process

#### State Change

Prop/State updates  
trigger reconciliation

#### Render Phase

Fiber builds workInProgress  
tree (interruptible)

#### Commit Phase

Apply changes to Virtual  
DOM (uninterruptible)

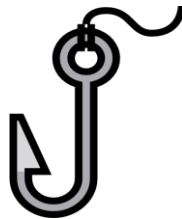
#### DOM Update

Minimal changes applied  
to the actual DOM



**NB** : Le state ne vit pas dans le composant, mais dans ce Fiber Node.

# React Navigation : React Hooks



**Hook** : Une fonction spéciale qui permet à React de mémoriser des valeurs entre les renders et de connecter ton composant au système interne de React (state, cycle de vie, context...).

Une fonction spéciale que React fournit pour donner des **super-pouvoirs** aux composants fonctionnels.

Ils permettent :

- d'avoir un état (**useState**)
- d'exécuter du code selon le cycle de vie du composant (**useEffect**)
- de mémoriser un calcul pour éviter de le refaire (**useMemo**)
- de mémoriser une valeur qui ne déclenche pas de re-render (**useRef**)
- d'accéder à un contexte global (**useContext**)
- d'optimiser les re-renders (**useCallback**, **useMemo**)

# Concepts de base de React Native : React Hooks

React stocke chaque Hook (useState, useEffect, etc.) dans une liste interne appelée memoizedState, située dans le Fiber du composant.

## Les règles des Hooks:

**R1 :** Utiliser les Hooks uniquement dans les composants fonctionnels.

**R2:** Appeler les Hooks uniquement au niveau supérieur du composant

**R3:** Toujours appeler les Hooks dans le même ordre

**R4:** Les Hooks doivent être appelés à chaque render

**R5:** Les Hooks personnalisés doivent commencer par use



## Map of Hooks

### State Management



useState



useReducer



useSyncExternalStore

### Context Hooks



useContext

### Transition Hooks



useTransition



useDeferredValue

### Ref Hooks



useRef



useImperativeHandle

### Random Hooks



useDebugValue



useId

### Performance Hooks



useMemo



useCallback

### Effect Hooks



useEffect



useLayoutEffect



useInsertionEffect

### React 19 Hooks



useFormStatus



useFormState



useOptimistic



use

# Concepts de base de React Native : React Hooks

## 1. useState

Permet à un composant fonctionnel de mémoriser une valeur et de se re-render quand elle change.

=> Quand le state change → React relance le composant → l'UI se met à jour.

```
export default function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <View style={{ padding: 20 }}>  
      <Text style={{ fontSize: 20 }}>  
        | Compteur : {count}  
      </Text>  
      <TouchableOpacity onPress={() => setCount(count + 1)}>  
        | <Text style={{ color: "white" }}> Incrémenter </Text>  
      </TouchableOpacity>  
    </View>  
  );  
}
```

À chaque changement de **count**, le composant est **re-rendu**.

# Concepts de base de React Native : React Hooks

## 2. useEffect

permet d'exécuter du code **APRÈS** le rendu du composant.

=> Tout ce qui n'est pas directement de l'affichage doit aller dans useEffect.

```
export default function ProfileScreen() {
  const [count, setcount]=useState(0);
  /* S'exécute à chaque render*/
  useEffect(() => {
    console.log("Le composant est affiché");
  });

  /*S'exécute une seule fois (au montage) */
  useEffect(() => {
    console.log("Le composant est affiché");
  }, []);

  /*S'exécute quand une valeur change */
  useEffect(() => {
    console.log("Le composant est affiché");
  }, [count]);
  return (
    <View style={{ flex: 1, padding: 20 }}>
      <Text style={{ fontSize: 24, fontWeight: "bold" }}>Profile</Text>
    </View>
  );
}
```

| Class component      | Hook                      |
|----------------------|---------------------------|
| componentDidMount    | useEffect() => {}, []     |
| componentDidUpdate   | useEffect() => {}, [deps] |
| componentWillUnmount | cleanup                   |

# Concepts de base de React Native : React Hooks

## 3. useRef

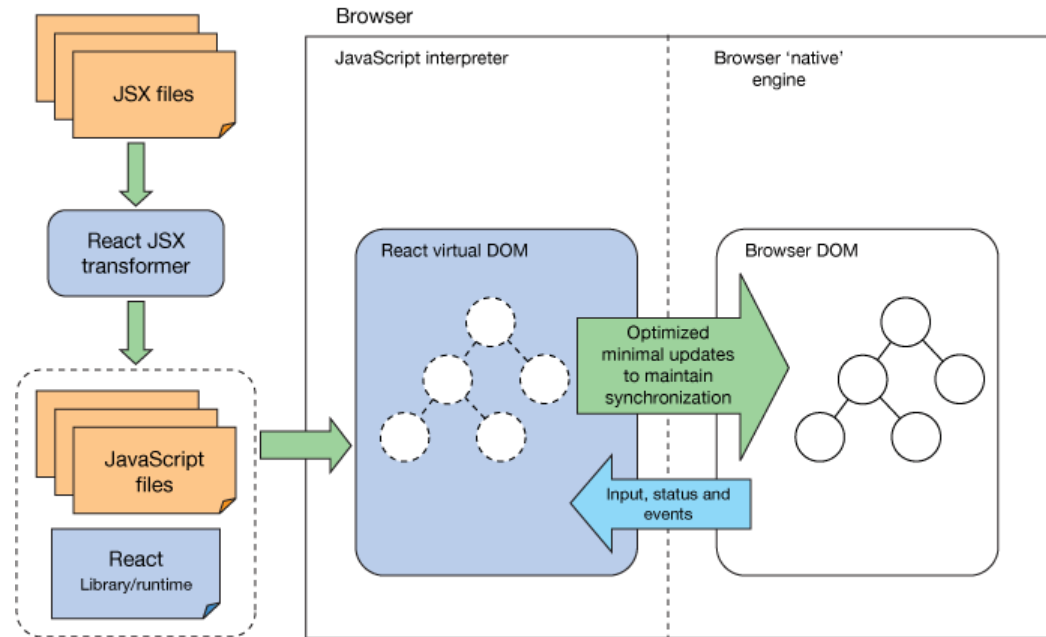
permet de garder une valeur en mémoire entre les renders  
=> sans déclencher de re-render quand cette valeur change.

```
export default function ProfileScreen() {  
  
  const clicks = useRef(0);  
  
  return (  
    <Button  
      title="Click"  
      onPress={() => {  
        clicks.current += 1;  
        console.log(clicks.current);  
      }}  
    />  
  );  
}
```

NB : La valeur change, l'UI ne change pas.

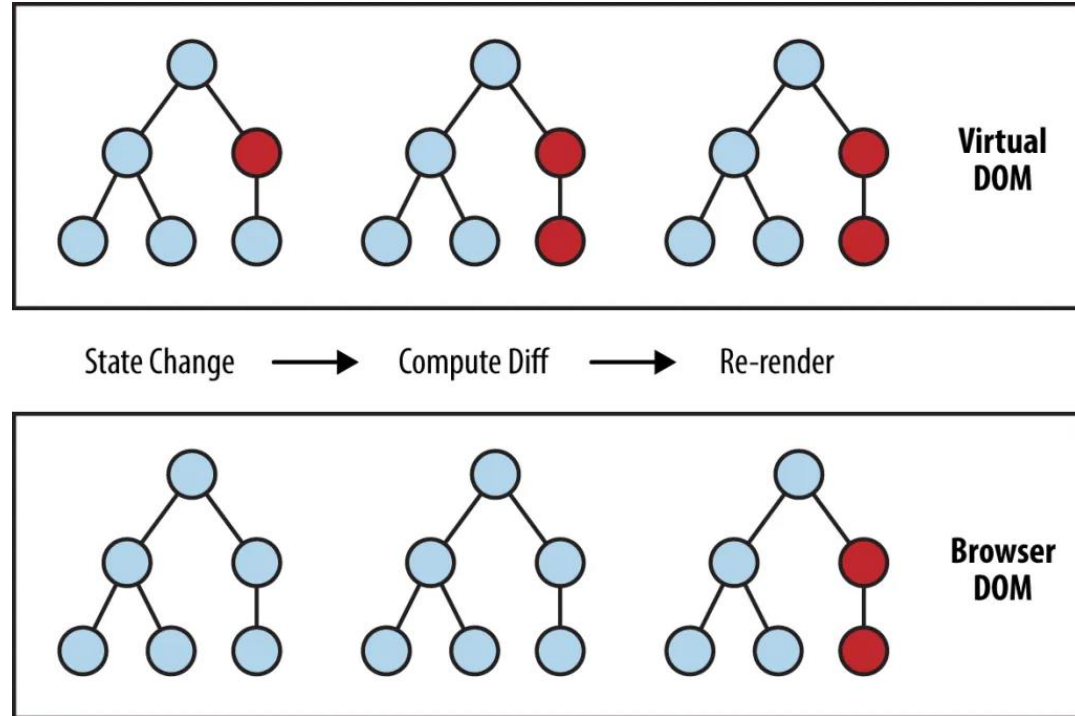
# React Component

- Le moteur de rendu JSX est une partie essentielle de React, qui transforme le code JSX en instructions compréhensibles par le moteur JavaScript pour créer et manipuler le DOM (Document Object Model).
- React utilise **Babel**, un transpilateur JavaScript, pour convertir JSX en JavaScript standard.
- Le moteur de React génère un DOM virtuel qui sera transformé et synchroniser avec le DOM réel du Navigateur.
- Le Virtual DOM optimise les changements à apporter au niveau du browser DOM. Ce qui lui permet d'accélérer l'aspect réactif du rendu.



# React Components : Virtual DOM

- Lorsqu'un élément change, le navigateur doit recalculer la mise en page (**reflow**) et redessiner (**repaint**) l'ensemble des éléments affectés.
- Cela peut entraîner un effet en cascade, où une modification sur un élément peut impacter plusieurs autres éléments.
- Modifier plusieurs éléments un par un entraîne de nombreux accès au DOM, ce qui ralentit l'application.
- Contrairement à un simple objet JavaScript en mémoire, le DOM est une interface complexe gérée par le navigateur, ce qui le rend plus lent à modifier



# Js coté serveur : Framework Nest JS

## Pourquoi un backend ?

- Notre application E-wallet en JavaScript natif fonctionne... mais seulement en apparence

## Problèmes du front-end seul

- Les données sont stockées dans le navigateur (sessionStorage, localStorage)
- Les données disparaissent si on change de navigateur ou d'appareil
- N'importe qui peut modifier le code côté client
- Aucune vraie sécurité
- Aucune gestion multi-utilisateurs réelle

Un front-end seul n'est pas fiable pour une application réelle.

# Js coté serveur : Framework Nest JS

Le backend est le cerveau et la mémoire de l'application.

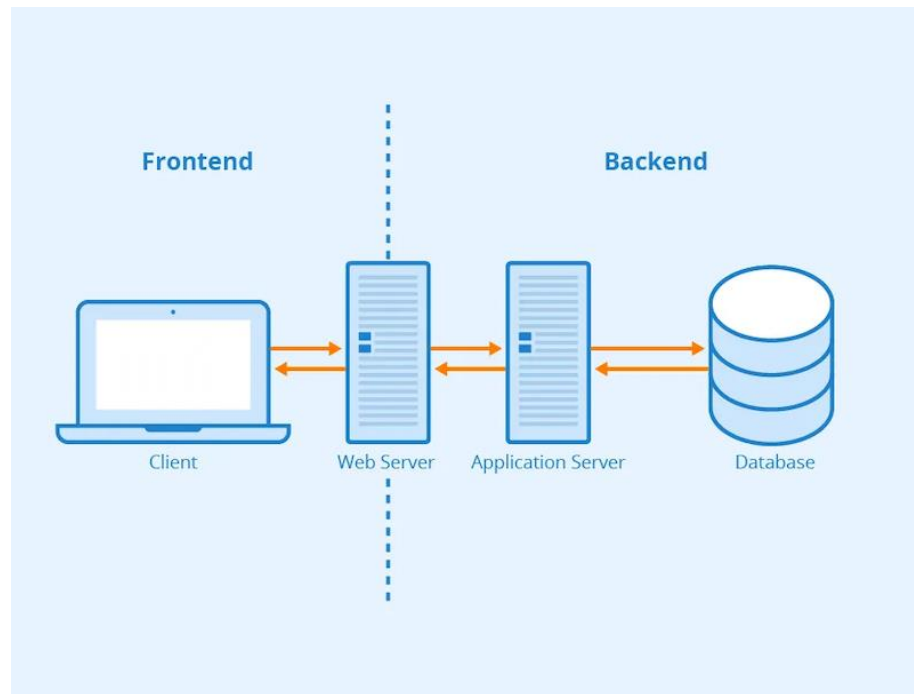
Il permet de :

- stocker les données de manière permanente
- appliquer les règles métier
- sécuriser les opérations
- gérer plusieurs utilisateurs simultanément

Le **backend** désigne tout ce qui fonctionne côté serveur.

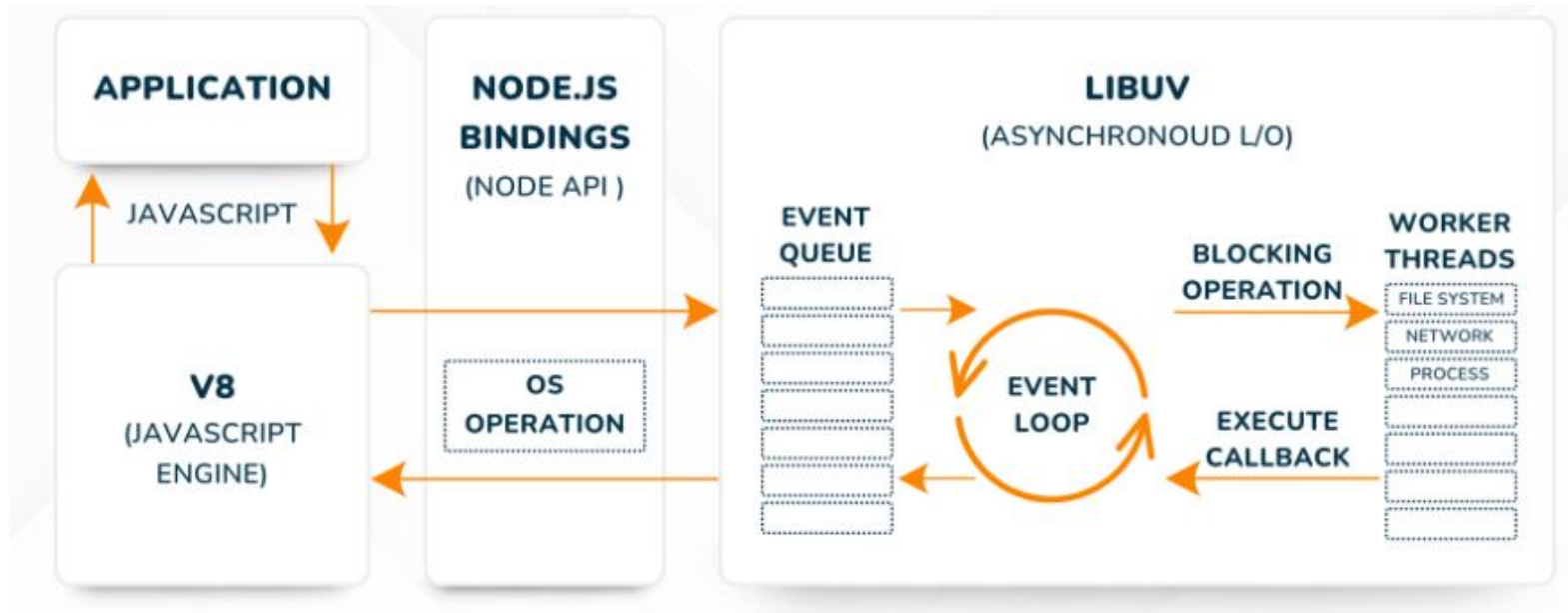
Il inclut généralement :

- un web server (gestion HTTP)
- un application server (logique métier)
- une base de données
- parfois des services externes (auth, paiement...)



## JavaScript côté serveur : Environnement Node.js

Node.js est un environnement d'exécution écrit en C++ qui permet d'exécuter du JavaScript côté serveur.



# JavaScript côté serveur : Environnement Node.js

installer l'environnement d'exécution Node.js

## Étape 1 : Télécharger Node.js (LTS)

- Accéder au site officiel de Node.js
- Télécharger la version LTS (Long Term Support)
- Lancer le programme d'installation

## Étape 2 : Installer Node.js

- Pendant l'installation :
- Conserver les options par défaut
- Vérifier que l'option "Add to PATH" est activée
- Laisser l'option "Automatically install the necessary tools..." désactivée (facultatif)

## Étape 3 : Vérifier l'installation

- Ouvrir un terminal (CMD, PowerShell ou Terminal)
- Vérifier les versions installées :

```
node -v  
npm -v
```

## npm (Node Package Manager)

npm est le gestionnaire de paquets officiel de Node.js.

Il permet de :

- Installer des bibliothèques
- Gérer les dépendances d'un projet
- Exécuter des scripts

# JavaScript côté serveur : Environnement Node.js

## Etape 4: Installation de NestJS

### 4.1 - Créer le dossier du projet

- Créer un dossier nommé **Back-end**
- Ce dossier contiendra tout le code côté serveur

### 4.2 - Ouvrir le dossier dans un éditeur de code

- Ouvrir le dossier Back-end avec un éditeur de code (exemple : Visual Studio Code)

### 4.3 - Ouvrir le terminal

- Ouvrir le terminal intégré de l'éditeur (Terminal → New Terminal)
- Vérifier que le terminal est bien positionné dans le dossier Back-end

### 4.4 - Installation et utilisation de la CLI NestJS

- **npx @nestjs/cli new Ewallet\_Backend**
- Cette commande exécute la CLI NestJS sans installation globale.
- Elle permet de générer automatiquement l'architecture initiale du projet.

### 4.5- Démarrer le serveur web NestJS

- **npm run start :dev**
- Lance le serveur en mode dev ( Redémarrage automatique lors des modifications du code)

# Introduction à NestJS

NestJS est un framework backend basé sur Node.js. Il utilise TypeScript et repose sur Express pour gérer les requêtes HTTP. Il propose une architecture **modulaire**, **l'injection de dépendances** et des **décorateurs** pour organiser le code. NestJS est un framework opinioné.



## Les Modules

- Un module regroupe les parties de l'application liées à une même fonctionnalité.
- Toute application NestJS contient au moins un module : le module racine (AppModule)

## Les Contrôleurs

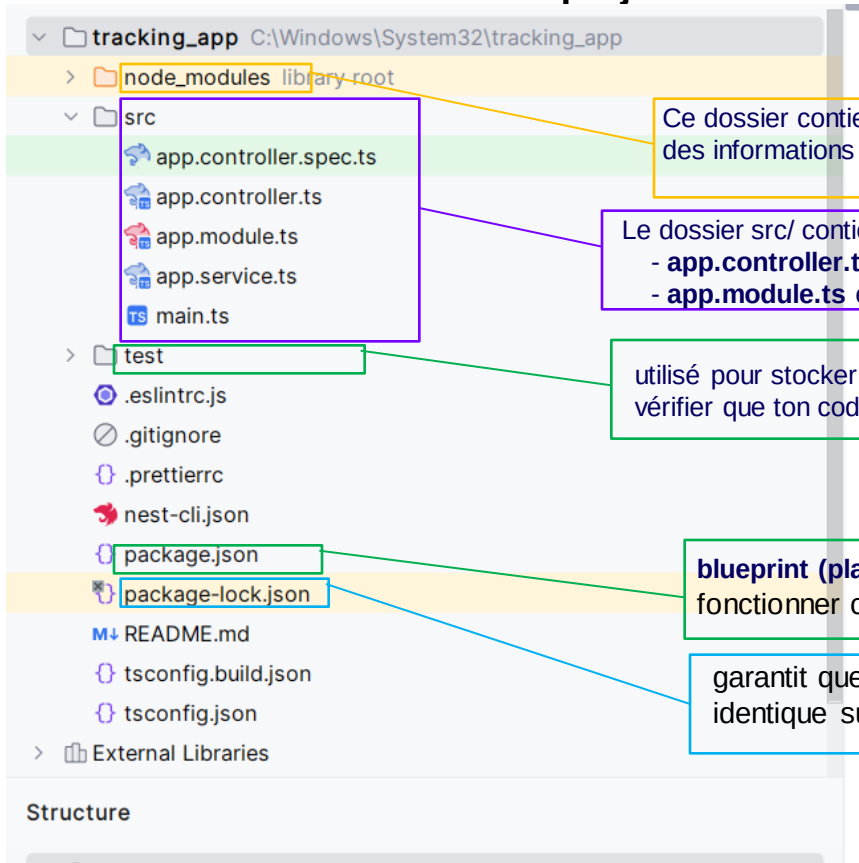
- Les contrôleurs reçoivent les requêtes HTTP et renvoient les réponses.
- Ils définissent les routes avec des décorateurs comme `@Get()`, `@Post()`, etc.

## Les Services (Providers)

- Les services contiennent la logique métier.
- Ils sont utilisés par les contrôleurs grâce à l'injection de dépendances.

# Back End Server Nestjs

## D- Vérifier la structure du projet :



Ce dossier contient toutes les dépendances installées par npm en fonction des informations de package.json.

Le dossier src/ contient le code source de l'application.  
- **app.controller.ts** contient le contrôleur principal de votre application.  
- **app.module.ts** est où vous configurez vos modules

utilisé pour stocker les fichiers de tests. Ces tests permettent de vérifier que ton code fonctionne correctement.

**blueprint (plan)** indique à **npm** ce dont ton projet a besoin pour fonctionner correctement.

garantit que les dépendances de ton projet seront installées de manière identique sur toutes les machines (ton ordinateur, ou sur un serveur).

# La structure de base d'une application NestJS

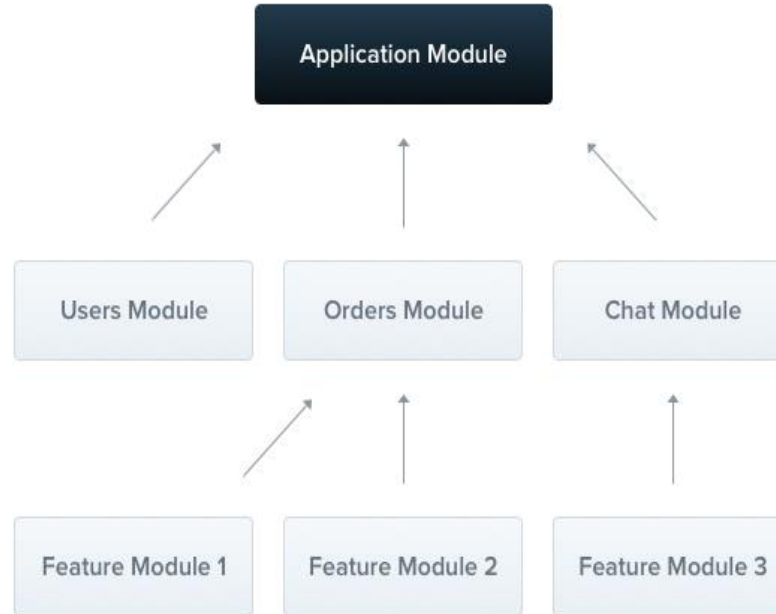


| Fichier           | Description                                                                                                                                                                                                                                                                | Exemple d'utilisation                                                                                                                                                                                                                                                                                     |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| app.controller.ts | Contrôleur de base avec <b>une seule route</b> . Il gère les requêtes HTTP entrantes et retourne les réponses appropriées.<br><br>- Utilise des décorateurs tels que @Controller et @Get (ainsi que d'autres méthodes HTTP comme @Post, @Put, etc.) pour définir une route | <pre>import { Controller, Get } from '@nestjs/common';<br/><br/>@Controller('example') // Prefix for the route: /example<br/>export class AppController {<br/>  @Get() // Handles GET requests to /example<br/>  getHello(): string {<br/>    return 'Hello, World!';<br/>  }<br/>}</pre>                 |
| app.module.ts     | Module racine de l'application. Organise les contrôleurs, les services et importe d'autres modules.<br>Un module est un regroupement logique de composants responsables d'une fonctionnalité ou d'une fonctionnalité spécifique de l'application.                          | <pre>import { Module } from '@nestjs/common';<br/><br/>@Module({<br/>  // Show usages<br/>  imports: [AuthentModule, UserModule],<br/>  controllers: [AppController, AuthController, UserController],<br/>  providers: [AppService, UserService, AuthService],<br/>})<br/>export class AppModule {}</pre> |
| app.service.ts    | Service de base avec une méthode simple. Contient la logique métier et est utilisé par les contrôleurs pour exécuter des tâches.                                                                                                                                           | <pre>import { Injectable } from '@nestjs/common';<br/><br/>@Injectable()<br/>export class AppService {<br/>  getHello(): string {<br/>    // Show usages<br/>    const text = 'Hello ILISI3';<br/>    return text;<br/>  }<br/>}</pre>                                                                    |
| main.ts           | Point d'entrée de l'application. Utilise NestFactory pour créer une instance de l'application NestJS et définir les configurations globales.                                                                                                                               | <pre>import { NestFactory } from '@nestjs/core';<br/><br/>async function bootstrap(): Promise&lt;void&gt; {<br/>  // Show usages<br/>  const app : INestApplication&lt;any&gt; = await NestFactory.create(AppModule);<br/>  await app.listen(process.env.PORT ?? 3000);<br/>}<br/><br/>bootstrap();</pre> |

# La structure de base d'une application NestJS



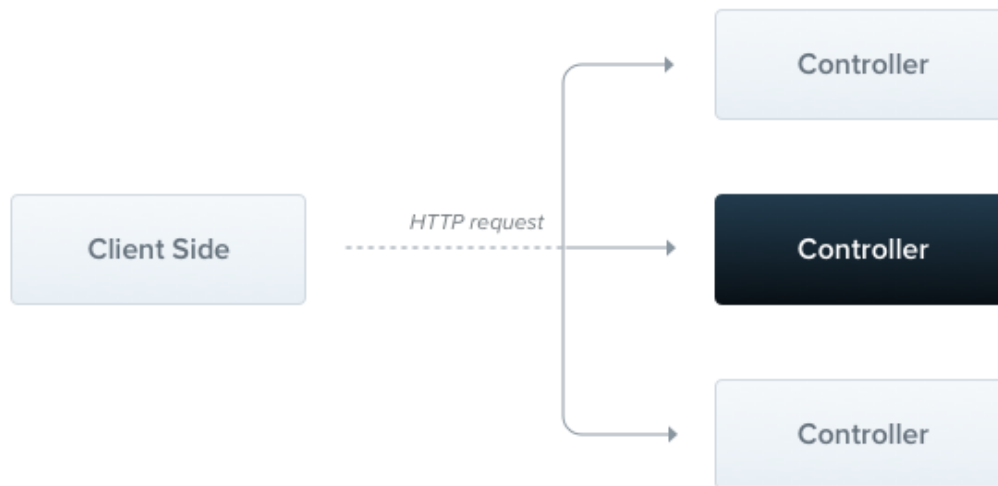
Le module racine constitue le point d'entrée de l'application NestJS. Il permet à Nest de construire un graphe interne décrivant les relations entre les modules et les fournisseurs afin de gérer l'injection des dépendances.



# La structure de base d'une application NestJS

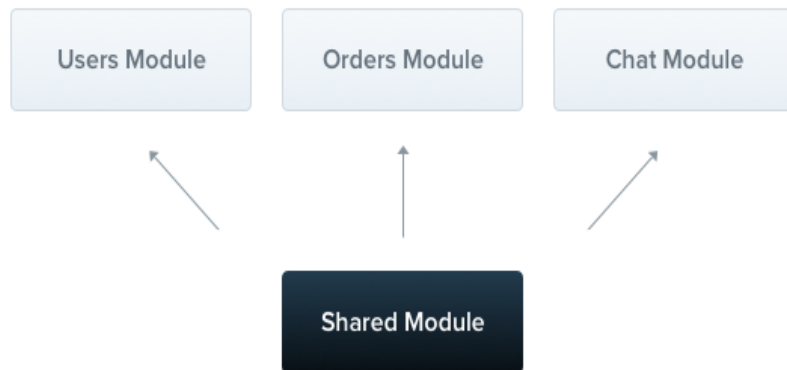
Les contrôleurs sont responsables du traitement des requêtes entrantes et de l'envoi des réponses au client.

Les contrôleurs reçoivent les requêtes HTTP, appellent les services appropriés et renvoient les réponses au client.



# La structure de base d'une application NestJS

Les providers d'un module sont des singletons par défaut. Pour les partager entre modules, ils doivent être exportés..

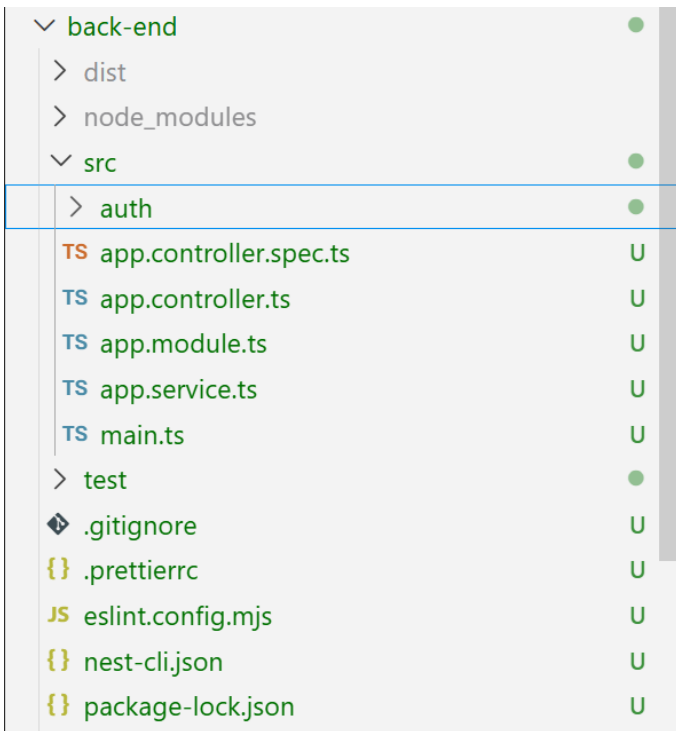


Chaque module est automatiquement un module partagé. Une fois créé, il peut être réutilisé par n'importe quel module.

# Back End Server Nest

## Étape 2 : Création des Modules et API (Module 1 : Authentification )

```
npx nest generate module auth  
npx nest generate controller auth  
npx nest generate service auth
```

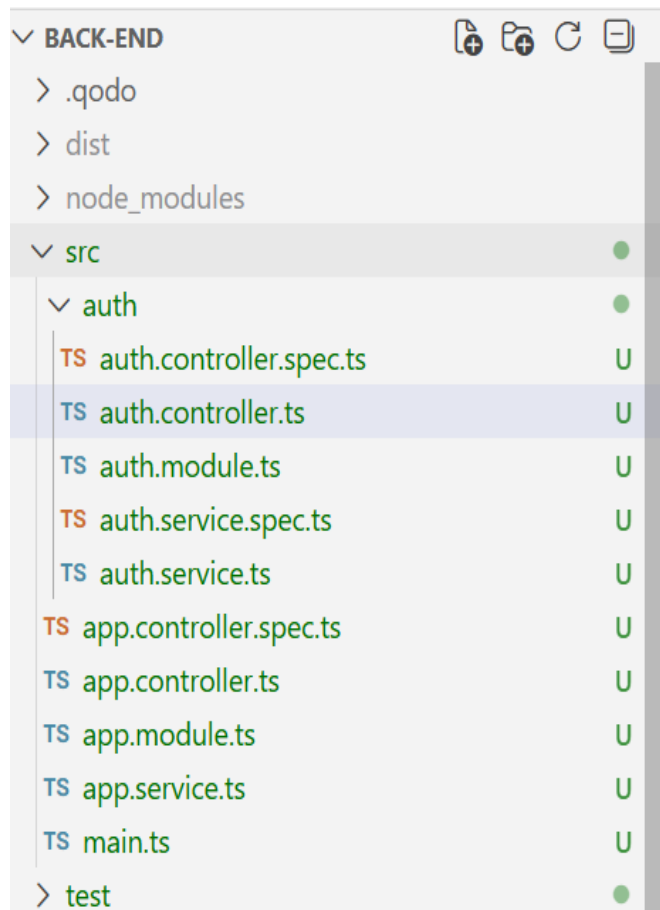


### auth.module.ts

```
import { Module } from '@nestjs/common';  
import { authController } from './auth.controller';  
import { authService } from './auth.service';
```

```
@Module({  
  controllers: [authController],  
  providers: [authService],  
})  
export class authModule {}
```

# Back End Server Nest



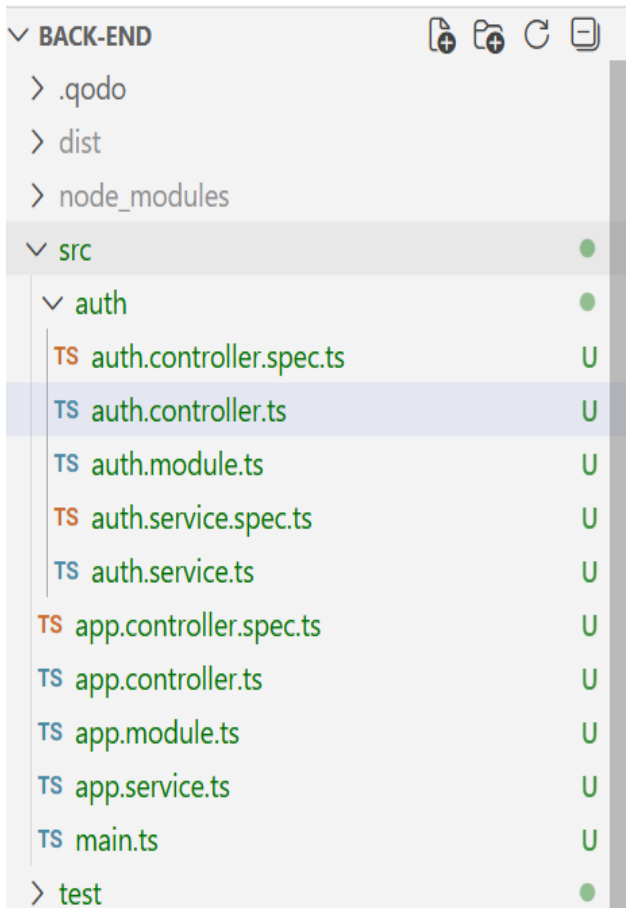
## Auth.controller.ts

```
import { Body, Controller, Get, Post, Query } from "@nestjs/common";
import { AuthService } from "../auth.service";
@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @Post('register')
  register(@Body() body: { email: string; password: string }): string {
    const { email, password } = body;
    return this.authService.register(email, password);
  }

  @Post('login')
  login(@Body() body: { email: string; password: string }) {
    console.log(body);
    const { email, password } = body;
    return this.authService.login(email, password);
  }
}
```

# Back End Server Nest



## Auth.service.ts

```
import { Injectable } from '@nestjs/common';
```

```
@Injectable()
```

```
export class AuthService {
```

```
  private users= []; // tableau pour stocker les utilisateur
```

```
  register(email: string , password : string) : string{
```

```
    this.users.push({email:email,password:password});
```

```
    return 'Utilisateur enregistré avec succès';
```

```
  }
```

```
  login(email: string , password : string) : string{
```

```
    const user = this.users.find(
```

```
      (user)=>user.email===email&& user.password===password);
```

```
    return user ? 'connexion reussie': 'cridentials invalide';
```

```
  }}
```

# Back End Server Nest

**fetch** est une API JavaScript native permettant d'envoyer des requêtes HTTP (GET, POST, PUT, DELETE...) vers un serveur.

- Intégrée au navigateur
- Basée sur les Promises
- Utilisée pour communiquer avec une API (backend)

## Syntaxe de base

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

## Syntaxe avec async (recommandé)

```
async function getData() {
  const response = await fetch(url);
  const data = await response.json();
  console.log(data);
}
```

## Exemple Fetch avec Post

```
fetch("url", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({login_data}),
});
```

# Back End Server Nest

- **JSON.stringify** transforme un objet JS en JSON
- **Content-Type** indique le format envoyé au serveur
- **fetch** retourne une Promise
- Une fois **résolue**, cette Promise donne un objet **Response**
- **response** est un objet HTTP, pas les données directement.

Il contient par exemple :

- response.status → code HTTP (200, 404, 500...)
- response.ok → true / false
- response.headers → en-têtes HTTP
- response.json() → méthode pour lire le body

Les données envoyées par le serveur ne sont pas directement accessibles.

## Exemple Fetch avec Post

```
const response = await fetch("http://localhost:3000/Ewallet/login", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify({ mail, password }),  
});  
console.log(response);
```

```
const response = await fetch(url);  
console.log(response.status); // 200  
console.log(response.ok);    // true  
const data = await response.json();  
console.log(data);
```

# Back End Server Nest

JSON (JavaScript Object Notation) est un format de données textuel, léger et lisible, utilisé pour échanger et stocker des données structurées.

## Caractéristiques

- Format texte
- Basé sur les objets JavaScript
- Facile à lire par l'humain
- Facile à parser par les machines

## Utilisé dans :

- APIs REST
- Frontend ↔ Backend
- Fichiers de configuration
- Stockage simple de données

## Structure d'un fichier JSON

```
{  
  "id": 1,  
  "email": "user@test.com",  
  "password": "1234",  
  "role": "USER",  
  "isActive": true  
}
```

## Types de données supportés

- string
- number
- boolean
- object
- array
- null

# Back End Server Nest

JSON (JavaScript Object Notation) est un format de données textuel, léger et lisible, utilisé pour échanger et stocker des données structurées.

## Lire un fichier JSON

```
import { promises as fs } from 'fs';
import * as path from 'path';

async getUsers() {
  const filePath = path.join(__dirname, '..', 'data', 'users.json');

  const data = await fs.readFile(filePath, 'utf-8');
  return JSON.parse(data);
}
```

**fs.readFile** → lit le fichier

**JSON.parse** → transforme le JSON en objet JavaScript

**path.join** → chemin portable

**import { promises as fs } from 'fs'** : On importe la version asynchrone du module fs (File System).  
**import \* as path from 'path'** ; Le module path permet de gérer les chemins de fichiers de façon portable.

process.cwd(): **Current Working Directory**

# Back End Server Nest

JSON (JavaScript Object Notation) est un format de données textuel, léger et lisible, utilisé pour échanger et stocker des données structurées.

## Ajouter des données (Ecriture)

```
async addUser(user) {  
  const filePath = path.join(__dirname, '..', 'data', 'users.json');  
  
  const data = await fs.readFile(filePath, 'utf-8');  
  const users = JSON.parse(data);  
  
  users.push(user);  
  
  await fs.writeFile(filePath, JSON.stringify(users, null, 2));  
}
```

## fs.writeFile(...)

Écrit des données dans un fichier.

- Si le fichier existe → il est écrasé
- S'il n'existe pas → il est créé

| Paramètre | Rôle                     |
|-----------|--------------------------|
| users     | Objet JS à convertir     |
| null      | Aucun filtre             |
| 2         | Indentation (lisibilité) |

# Installation de PostgreSQL

## 1. Télécharger PostgreSQL

<https://www.postgresql.org/download/>

Choisis le système d'exploitation (windows, linux, macOS)



## 2. Installation sous Windows

Étapes :

1. Lancer le fichier .exe
2. Cliquer sur Next
3. Choisir le dossier d'installation (laisser par défaut)
4. Sélectionner les composants (recommandé) :
  - PostgreSQL Server
  - pgAdmin 4
  - Command Line Tools

## 5. Définir le mot de passe du super-utilisateur postgres

- Note-le bien (important pour PostGIS, pgAdmin, TypeORM...)
- Port : 5432 (par défaut, ne pas changer)
- Locale : Default
- Install → Finish

# NestJs : PostgreSQL avec TypeORM

## Etape 1 : Installer les dépendances nécessaires

installer les paquets nécessaires pour TypeORM et PostgreSQL

```
npm install @nestjs/typeorm typeorm pg
```

## Etape 2 : Configurer TypeORM :

configurer la connexion à la base de données et déclarer les entités.

```
6 import { TypeOrmModule } from "@nestjs/typeorm";
7 import { User } from './user/entities/user.entity';
  Qodo: Test this class
8 @Module({
9   imports: [
10    //configuration de TypeORM
11    TypeOrmModule.forRoot({
12      type: 'postgres',
13      host: 'localhost',
14      port: 5433,
15      username: 'postgres',
16      password: 'admin',
17      database: 'BusTracking_db',
18      entities: [User],
19      synchronize: true, // DEV uniquement
20    }),AuthModule, UserModule],
21    controllers: [AppController],
22    providers: [AppService],
23  })
```

**TypeOrmModule.forRoot** configure la connexion à la base de données et doit être importé une seule fois dans le module **racine** de l'application.

l'option **synchronize** met automatiquement à jour le schéma de la base de données à partir des entités. Elle est utile en phase de développement, mais dangereuse en environnement de production.

# NestJs : TypeORM

## Etape 3 : Créer une Entity

```
back-end > src > user > entities > TS user.entity.ts > 🐞 User > 🔗 fullName
1  import { Entity, PrimaryGeneratedColumn,
2  |      |      Column, CreateDateColumn, } from "typeorm";
   Qodo: Test this class
3  @Entity("users")
4  export class User {
5      @PrimaryGeneratedColumn()
6      id: number;
7      @Column({ unique: true })
8      email: string;
9      @Column()
10     ⚡ password: string;
11     @Column()
12     fullName: string;
13     @Column()
14     role: string;
15     @CreateDateColumn()
16     createdAt: Date;
17 }
```

Un module peut contenir plusieurs entités,  
tant qu'elles appartiennent au même domaine  
métier

## Etape 4: Importer l'Entity dans son Module

Rendre l'entité exploitable via un Repository,  
puis l'utiliser dans un service

```
back-end > src > user > TS user.module.ts > ...
1  import { Module } from '@nestjs/common';
2  import { UserController } from '../user.controller';
3  import { UserService } from '../user.service';
4  import { TypeOrmModule } from '@nestjs/typeorm';
5  import { User } from '../entities/user.entity';
6
   Qodo: Test this class
7  @Module({
8      imports: [TypeOrmModule.forFeature([User])],
9      controllers: [UserController],
10     providers: [UserService]
11 })
12 export class UserModule {}
```

**forFeature([User])** : Active le repository  
(Repository<User>) d'une entité dans un module

# NestJs : TypeORM

## Etape 5: Injection du Repository

Injecter le repository dans un service et implémenter les premières méthodes CRUD

back-end > src > user > TS user.service.ts > UsersService > constructor

```
1 import { Injectable } from '@nestjs/common';
2 import { Repository } from "typeorm";
3 import { InjectRepository } from "@nestjs/typeorm";
4 import { User } from "../entities/user.entity";
5
6 Qodo: Test this class
7 @Injectable()
8 export class UsersService {
9   constructor(
10     @InjectRepository(User)
11     private readonly userRepository: Repository<User>
12   ) {}
13 }
```

- **@InjectRepository(User)** : demande à NestJS le repository activé

- **Repository<User>** est l'API CRUD fournie par TypeORM

- Le service est le seul endroit qui parle à la DB

## Étape 6 — Écrire les méthodes CRUD de base

- create() → crée l'objet (mémoire)
- save() → INSERT/UPDATE
- findOne() → SELECT
- find() → SELECT \*

```
create(user: Partial<User>) {
  const entity = this.userRepository.create(user);
  return this.userRepository.save(entity);
}

// Trouver un utilisateur par email
Qodo: Test this method
findByEmail(email: string) {
  return this.userRepository.findOne({ where: { email } });
}
```

# NestJs : Les relations TypeORM

## Objectif

Relier des objets entre eux comme dans le monde réel.

## Exemples réels

- Un User possède plusieurs Accounts
- Un Account contient plusieurs Transactions
- Une Transaction appartient à un seul Account

| Relation   | Signification |
|------------|---------------|
| OneToOne   | 1 ↔ 1         |
| OneToMany  | 1 → N         |
| ManyToOne  | N → 1         |
| ManyToMany | N ↔ N         |

Les relations permettent de modéliser ces liens

# NestJs : Les relations TypeORM

## Relation OneToMany / ManyToOne (la plus utilisée)

### Exemple métier

- Un User peut avoir plusieurs Accounts
- Un Account appartient à un seul User

@OneToMany prend deux fonctions callback :

- la première précise l'entité du côté **Many**
- la deuxième précise la propriété inverse qui pointe vers le **One**

```
@OneToMany( () => Account, (account) => account.user, {  
  cascade: true,}  
accounts: Account[];
```

```
@ManyToOne( () => user, (user) => user.account)
```

```
user: User;
```

Un User possède plusieurs Account,  
et chaque Account référence son User via la propriété user.

| Relation   | Signification |
|------------|---------------|
| OneToOne   | 1 ↔ 1         |
| OneToMany  | 1 → N         |
| ManyToOne  | N → 1         |
| ManyToMany | N ↔ N         |

